

1. Exploratory Data Analysis (EDA)

1.1 Objective

The goal of this analysis is to explore the ICBC crash dataset (<https://www.kaggle.com/datasets/tcashion/icbc-vehicle-crash-dataset/data>), understand its structure and quality, identify patterns related to casualty crashes, and determine whether the available variables are suitable for predicting high-risk intersections.

This EDA will:

- Examine data quality and completeness
- Understand the distribution of casualty crashes
- Explore temporal and geographic patterns
- Investigate relationships between crash characteristics and crash severity
- Generate observations that guide feature engineering and model selection

```
In [1]: # imports
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import mean_absolute_error, mean_squared_error

plt.style.use("ggplot")
pd.set_option("display.max_columns", None)
```

```
In [2]: with open("British_Columbia_Full_Data_data.csv", "rb") as f:
        print(f.read(16))

# Load dataset
df = pd.read_csv(
    "British_Columbia_Full_Data_data.csv", encoding="utf-16", sep="\t", low_memory=
)

# dataset shape and head
print(df.shape)
df.head()
```

```
b'\xff\xfeC\x00r\x00a\x00s\x00h\x00 \x00B\x00'
(1380474, 29)
```

Out[2]:

	Crash Breakdown 2	Date Of Loss Year	Animal Involved	Crash Severity	Cyclist Involved	Day Of Week	Derived Crash Congifuration	Heavy Veh Involved	Inte
0	SURREY	2018	No	CASUALTY CRASH	No	FRIDAY	REAR END	No	
1	SURREY	2018	No	CASUALTY CRASH	No	FRIDAY	REAR END	No	
2	SURREY	2018	No	CASUALTY CRASH	No	FRIDAY	CONFLICTED	No	
3	COQUITLAM	2018	No	CASUALTY CRASH	No	FRIDAY	REAR END	No	
4	SURREY	2018	No	CASUALTY CRASH	No	FRIDAY	REAR END	No	



In [3]:

```
# dataset info
print(df.shape)
df.info()
```

```

(1380474, 29)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1380474 entries, 0 to 1380473
Data columns (total 29 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Crash Breakdown 2                         1380474 non-null object
1   Date Of Loss Year                         1380474 non-null int64
2   Animal Involved                           1380474 non-null object
3   Crash Severity                           1380474 non-null object
4   Cyclist Involved                         1380474 non-null object
5   Day Of Week                             1380474 non-null object
6   Derived Crash Congifuration              1380474 non-null object
7   Heavy Veh Involved                       1380474 non-null object
8   Intersection Crash                       1380474 non-null object
9   Month Of Year                            1380474 non-null object
10  Motorcycle Involved                      1380474 non-null object
11  Municipality Name (ifnull)               1380474 non-null object
12  Parked Vehicle Involved                  1380474 non-null object
13  Parking Lot Crash                       1380474 non-null object
14  Pedestrian Involved                     1380474 non-null object
15  Region                                  1380474 non-null object
16  Street Full Name (ifnull)                1380283 non-null object
17  Time Category                           1380474 non-null object
18  Municipality Name                       1359752 non-null object
19  Road Location Description                 1380296 non-null object
20  Street Full Name                        1377647 non-null object
21  Metric Selector                         1380474 non-null int64
22  Total Crashes                           1380474 non-null int64
23  Total Victims                           1380474 non-null int64
24  Cross Street Full Name                   640013 non-null object
25  Latitude                                1087337 non-null float64
26  Longitude                                1087337 non-null float64
27  Mid Block Crash                         1380474 non-null object
28  Municipality With Boundary               1087337 non-null object
dtypes: float64(2), int64(4), object(23)
memory usage: 305.4+ MB

```

```

In [4]: # dataset columns
df.columns.tolist()

```

```
Out[4]: ['Crash Breakdown 2',
         'Date Of Loss Year',
         'Animal Involved',
         'Crash Severity',
         'Cyclist Involved',
         'Day Of Week',
         'Derived Crash Congifuration',
         'Heavy Veh Involved',
         'Intersection Crash',
         'Month Of Year',
         'Motorcycle Involved',
         'Municipality Name (ifnull)',
         'Parked Vehicle Involved',
         'Parking Lot Crash',
         'Pedestrian Involved',
         'Region',
         'Street Full Name (ifnull)',
         'Time Category',
         'Municipality Name',
         'Road Location Description',
         'Street Full Name',
         'Metric Selector',
         'Total Crashes',
         'Total Victims',
         'Cross Street Full Name',
         'Latitude',
         'Longitude',
         'Mid Block Crash',
         'Municipality With Boundary']
```

```
In [5]: # missing values
missing = df.isnull().sum().sort_values(ascending=False)

missing = missing[missing > 0]

missing
```

```
Out[5]: Cross Street Full Name      740461
Municipality With Boundary      293137
Longitude                        293137
Latitude                        293137
Municipality Name                20722
Street Full Name                 2827
Street Full Name (ifnull)         191
Road Location Description         178
dtype: int64
```

```
In [6]: # dataset missing values percentage
(df.isnull().mean().mul(100).sort_values(ascending=False).round(2))
```

```
Out[6]: Cross Street Full Name      53.64
Municipality With Boundary      21.23
Longitude                       21.23
Latitude                       21.23
Municipality Name                1.50
Street Full Name                 0.20
Street Full Name (ifnull)       0.01
Road Location Description       0.01
Region                          0.00
Mid Block Crash                 0.00
Total Victims                   0.00
Total Crashes                   0.00
Metric Selector                 0.00
Time Category                   0.00
Crash Breakdown 2               0.00
Date Of Loss Year               0.00
Parking Lot Crash               0.00
Parked Vehicle Involved         0.00
Municipality Name (ifnull)      0.00
Motorcycle Involved             0.00
Month Of Year                   0.00
Intersection Crash              0.00
Heavy Veh Involved              0.00
Derived Crash Congifuration     0.00
Day Of Week                     0.00
Cyclist Involved                0.00
Crash Severity                  0.00
Animal Involved                 0.00
Pedestrian Involved             0.00
dtype: float64
```

```
In [7]: # duplicate rows
print(df.duplicated().sum())
```

0

Intersection data QA

```
In [8]: # handle intersection names
def normalize_intersection(row):
    street1 = str(row["Street Full Name (ifnull)"]).strip().upper()
    street2 = str(row["Cross Street Full Name"]).strip().upper()

    streets = sorted([street1, street2])

    return " @ ".join(streets)
```

Vancouver specific DF

```
In [9]: vancouver_df = df[df["Municipality Name (ifnull)"] == "VANCOUVER"].copy()

print(vancouver_df.shape)
```

(225148, 29)

```
In [10]: # keep valid Vancouver intersection crashes
intersection_df = vancouver_df[
    (vancouver_df["Intersection Crash"] == "Yes")
    & (vancouver_df["Cross Street Full Name"].notna())
].copy()
```

```
In [11]: intersection_df["Intersection"] = intersection_df.apply(normalize_intersection, axis=1)
```

```
In [12]: original_intersections = (
    intersection_df["Street Full Name (ifnull)"].str.strip().str.upper()
    + " @ "
    + intersection_df["Cross Street Full Name"].str.strip().str.upper()
)

print("Original unique intersection IDs:", original_intersections.nunique())
print("Normalized unique intersections:", intersection_df["Intersection"].nunique())
print(
    "Duplicate directional IDs consolidated:",
    original_intersections.nunique() - intersection_df["Intersection"].nunique(),
)
```

Original unique intersection IDs: 9363

Normalized unique intersections: 5617

Duplicate directional IDs consolidated: 3746

Intersection identifiers were constructed from the two street names and normalized by alphabetically sorting the street pair. This prevents the same physical intersection from being treated as separate locations when the street order is reversed (e.g., BOUNDARY RD @ GRANDVIEW HWY and GRANDVIEW HWY @ BOUNDARY RD). Normalization reduced the number of unique intersection identifiers from 9,363 to 5,617, consolidating 3,746 duplicate directional identifiers.

```
In [13]: # vancouver dataset missing values percentage
(vancouver_df.isnull().mean().mul(100).sort_values(ascending=False).round(2))
```

```
Out[13]: Cross Street Full Name      38.33
Municipality With Boundary      10.20
Longitude                       10.20
Latitude                       10.20
Street Full Name                 0.16
Street Full Name (ifnull)       0.01
Road Location Description       0.00
Region                          0.00
Mid Block Crash                 0.00
Total Victims                   0.00
Total Crashes                   0.00
Metric Selector                 0.00
Municipality Name               0.00
Time Category                   0.00
Crash Breakdown 2               0.00
Date Of Loss Year               0.00
Parking Lot Crash               0.00
Parked Vehicle Involved         0.00
Municipality Name (ifnull)      0.00
Motorcycle Involved             0.00
Month Of Year                   0.00
Intersection Crash              0.00
Heavy Veh Involved              0.00
Derived Crash Congifuration     0.00
Day Of Week                     0.00
Cyclist Involved                0.00
Crash Severity                  0.00
Animal Involved                 0.00
Pedestrian Involved             0.00
dtype: float64
```

Crash Severity

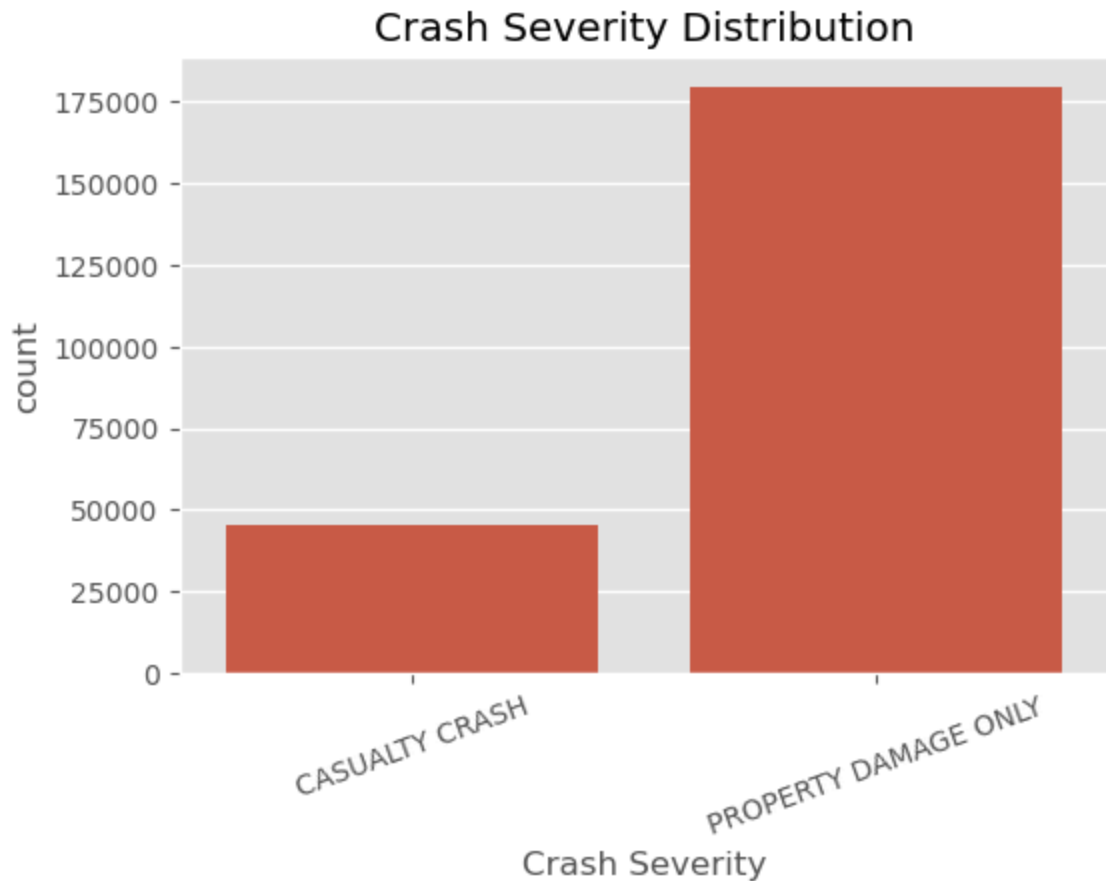
```
In [14]: # target variable distribution

plt.figure(figsize=(6, 4))

sns.countplot(data=vancouver_df, x="Crash Severity")

plt.title("Crash Severity Distribution")
plt.xticks(rotation=20)

plt.show()
```

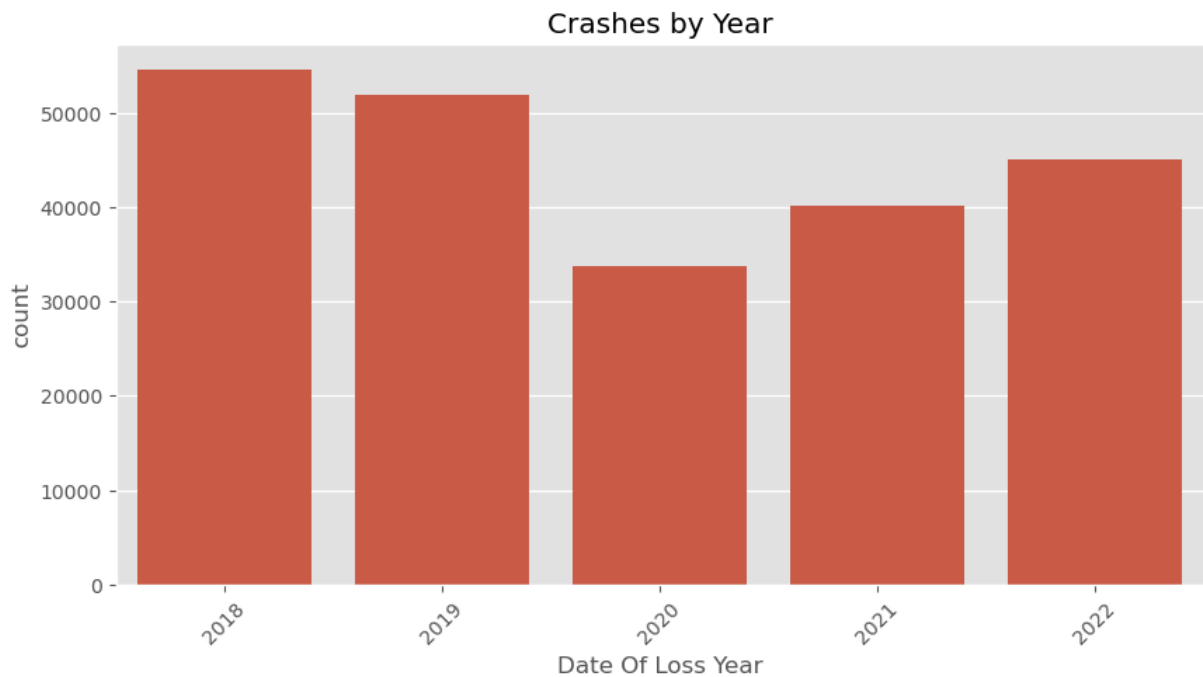


```
In [15]: # Target variable percentages  
(vancouver_df["Crash Severity"].value_counts(normalize=True).mul(100).round(2))
```

```
Out[15]: Crash Severity  
PROPERTY DAMAGE ONLY    79.8  
CASUALTY CRASH          20.2  
Name: proportion, dtype: float64
```

Temporal exploration

```
In [16]: # crashes by year  
plt.figure(figsize=(10, 5))  
  
sns.countplot(data=vancouver_df, x="Date Of Loss Year")  
  
plt.xticks(rotation=45)  
  
plt.title("Crashes by Year")  
plt.show()
```

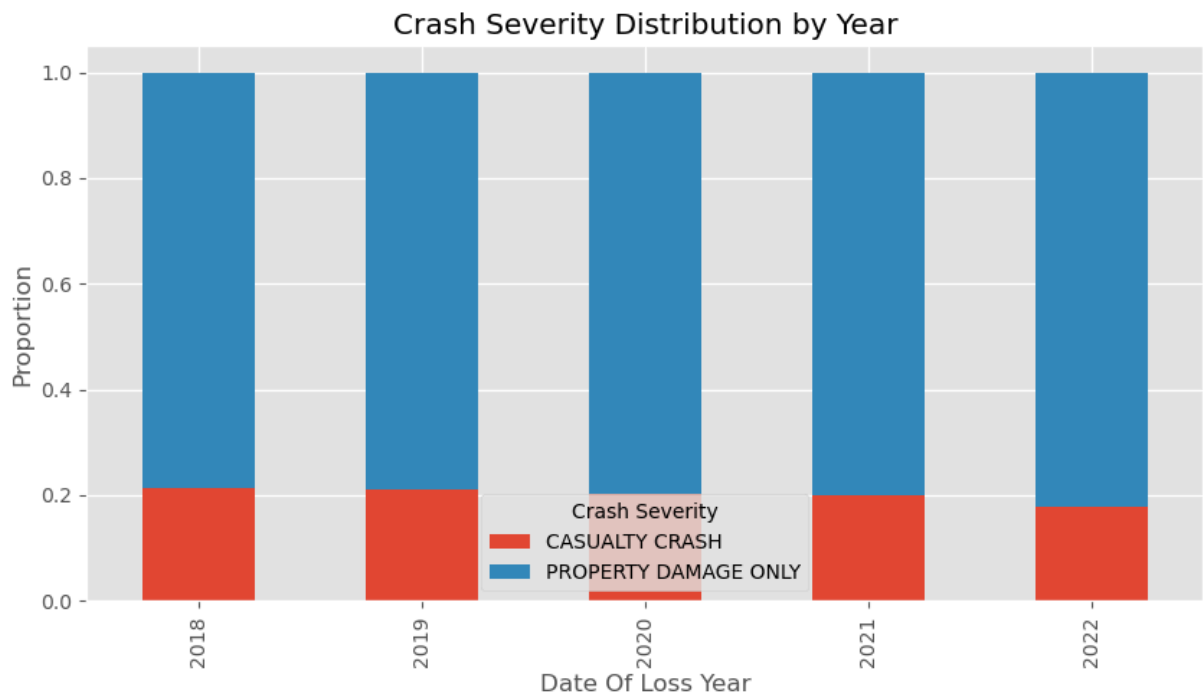
```
In [17]: year_severity = pd.crosstab(
    vancouver_df["Date Of Loss Year"], vancouver_df["Crash Severity"], normalize="i
)

display(year_severity)

year_severity.plot(kind="bar", stacked=True, figsize=(10, 5))

plt.ylabel("Proportion")
plt.title("Crash Severity Distribution by Year")
plt.show()
```

Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Date Of Loss Year		
2018	0.214134	0.785866
2019	0.211301	0.788699
2020	0.203017	0.796983
2021	0.199631	0.800369
2022	0.178117	0.821883



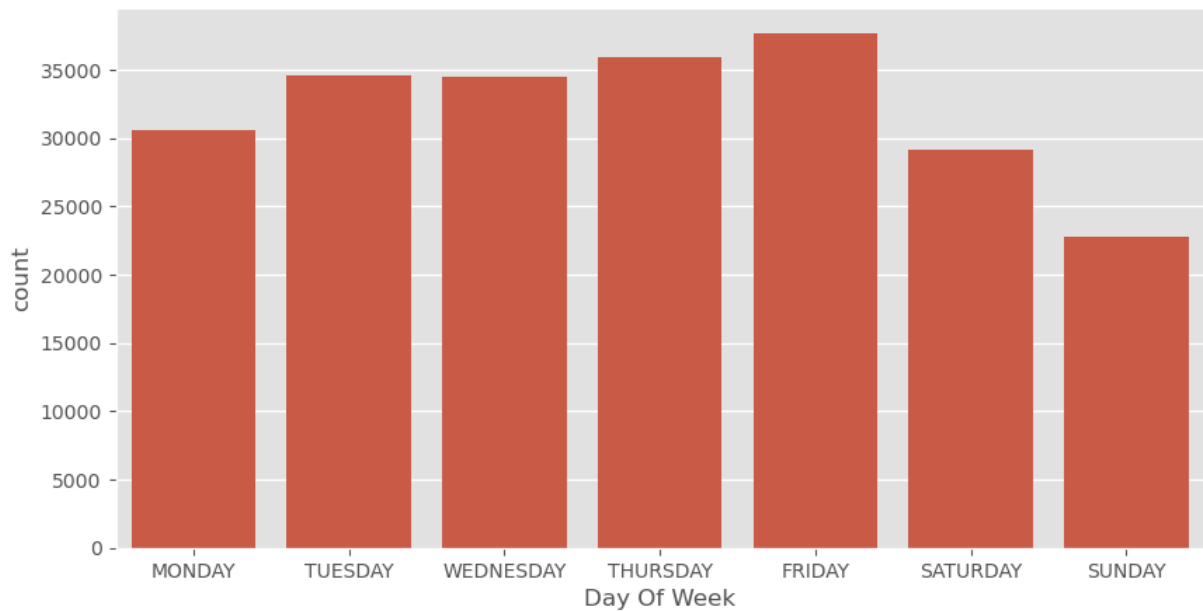
```
In [18]: # crashes by day of the week

order = ["MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY", "SATURDAY", "SUNDAY"]

plt.figure(figsize=(10, 5))

sns.countplot(data=vancouver_df, x="Day Of Week", order=order)

plt.show()
```



```
In [19]: # crashes by month

month_order = [
    "JANUARY",
    "FEBRUARY",
```

```

    "MARCH",
    "APRIL",
    "MAY",
    "JUNE",
    "JULY",
    "AUGUST",
    "SEPTEMBER",
    "OCTOBER",
    "NOVEMBER",
    "DECEMBER",
]

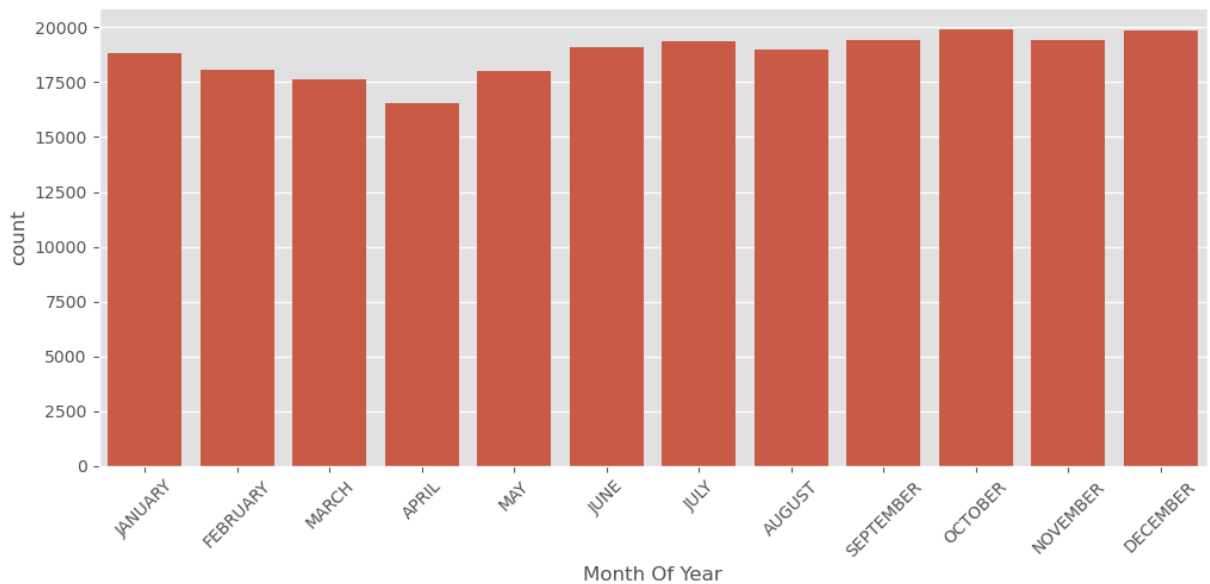
plt.figure(figsize=(12, 5))

sns.countplot(data=vancouver_df, x="Month Of Year", order=month_order)

plt.xticks(rotation=45)

plt.show()

```



```

In [20]: time_rates = pd.crosstab(
    vancouver_df["Time Category"], vancouver_df["Crash Severity"]
)

time_order = [
    "00:00-02:59",
    "03:00-05:59",
    "06:00-08:59",
    "09:00-11:59",
    "12:00-14:59",
    "15:00-17:59",
    "18:00-20:59",
    "21:00-23:59",
]

time_rates = time_rates.reindex(time_order)[["CASUALTY CRASH", "PROPERTY DAMAGE ONLY", "FATALITY CRASH", "PROPERTY DAMAGE ONLY AND FATALITY CRASH"]]

```

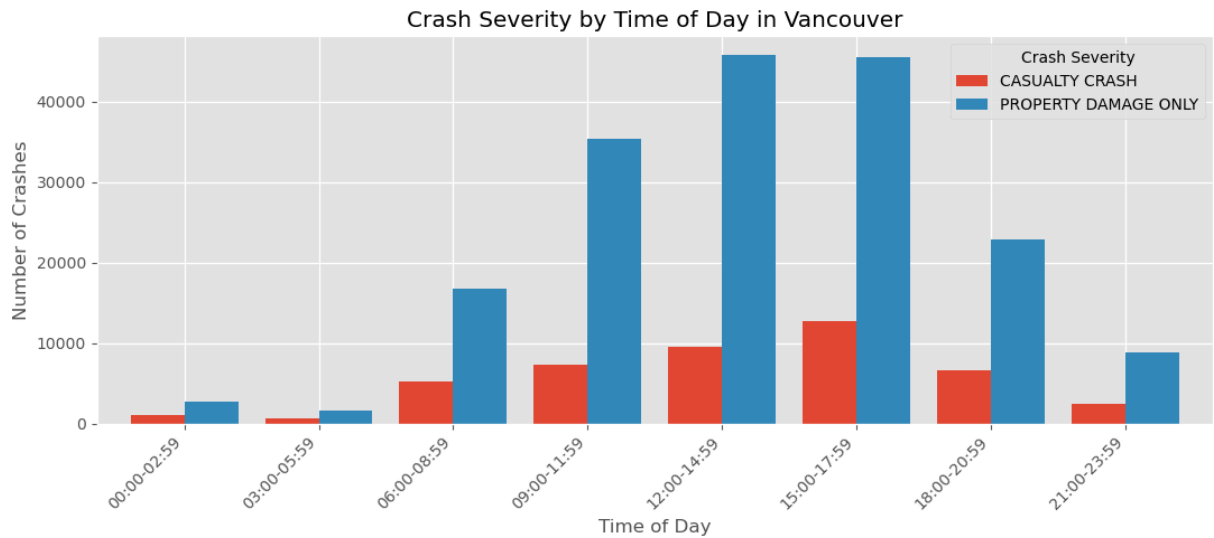
```

ax = time_rates.plot(kind="bar", stacked=False, figsize=(11, 5), width=0.8)

ax.set_title("Crash Severity by Time of Day in Vancouver")
ax.set_xlabel("Time of Day")
ax.set_ylabel("Number of Crashes")
ax.legend(title="Crash Severity")

plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```



```

In [21]: time_rates = pd.crosstab(vancouver_df["Time Category"], vancouver_df["Crash Severity"]

time_order = [
    "00:00-02:59",
    "03:00-05:59",
    "06:00-08:59",
    "09:00-11:59",
    "12:00-14:59",
    "15:00-17:59",
    "18:00-20:59",
    "21:00-23:59",
]

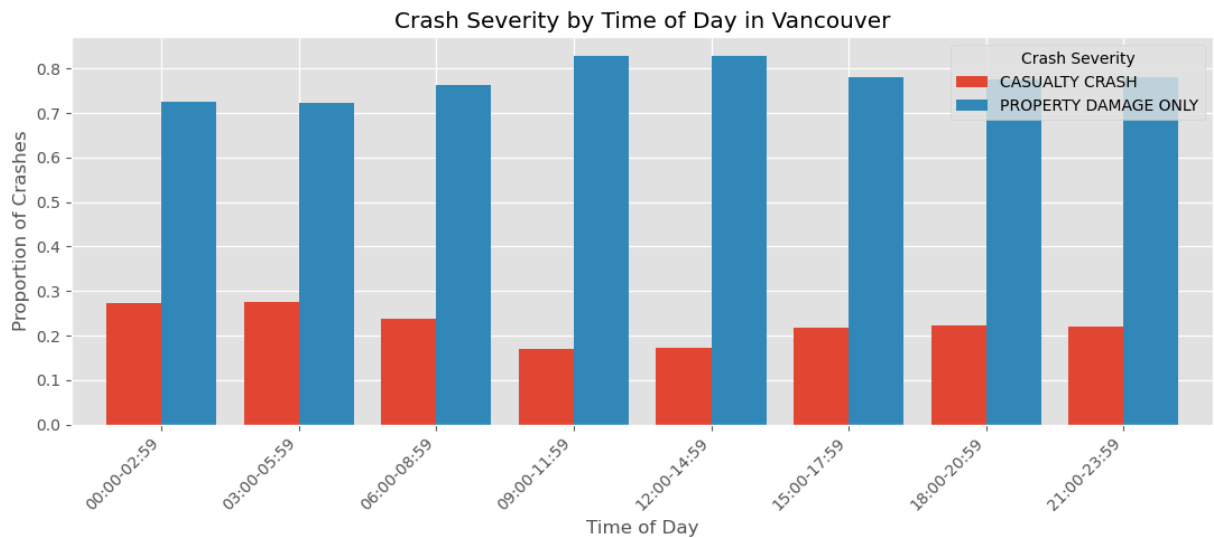
time_rates = time_rates.reindex(time_order)[["CASUALTY CRASH", "PROPERTY DAMAGE ONLY"]

ax = time_rates.plot(kind="bar", stacked=False, figsize=(11, 5), width=0.8)

ax.set_title("Crash Severity by Time of Day in Vancouver")
ax.set_xlabel("Time of Day")
ax.set_ylabel("Proportion of Crashes")
ax.legend(title="Crash Severity")

plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```

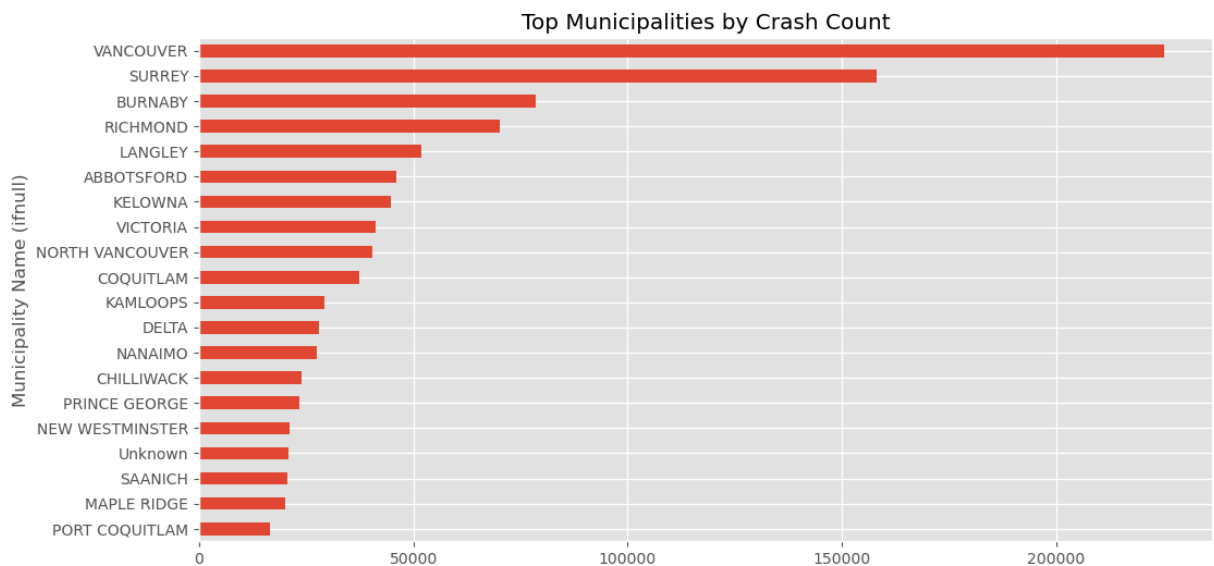


BC Geographic analysis

```
In [22]: # crashes by municipality
plt.figure(figsize=(12, 6))

(df["Municipality Name (ifnull)"].value_counts().head(20).sort_values().plot.barh())

plt.title("Top Municipalities by Crash Count")
plt.show()
```

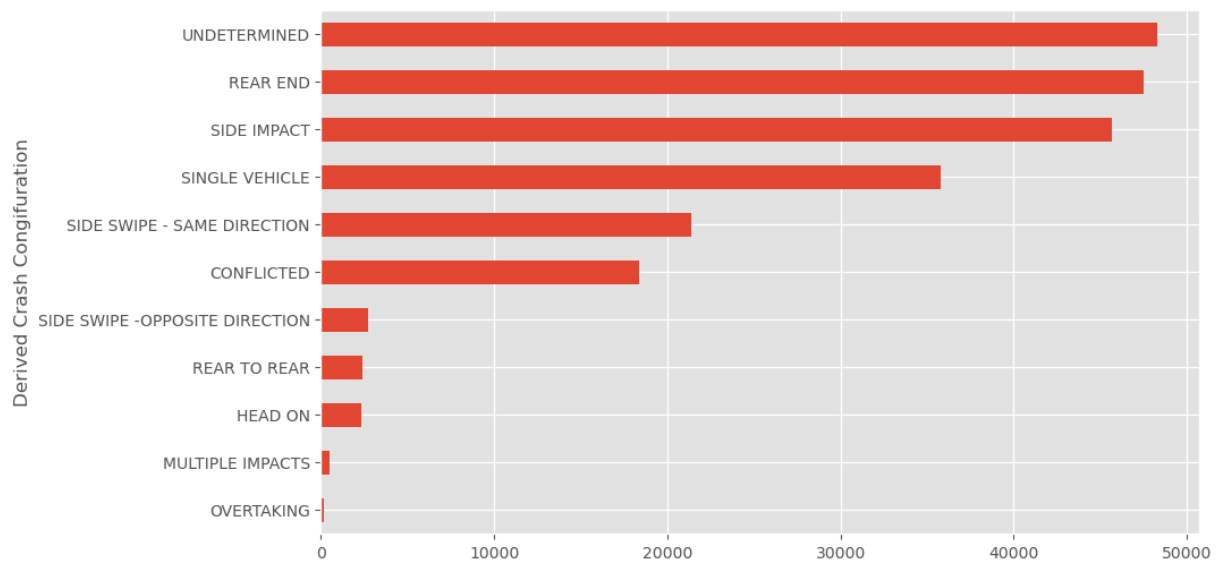


Vancouver Crash characteristics

```
In [23]: # crashes by derived crash configuration
plt.figure(figsize=(10, 6))

(vancouver_df["Derived Crash Congifuration"].value_counts().head(15).sort_values().

plt.show()
```



Vancouver Binary crash variables

```
In [24]: binary_cols = [  
    "Animal Involved",  
    "Cyclist Involved",  
    "Heavy Veh Involved",  
    "Motorcycle Involved",  
    "Pedestrian Involved",  
    "Intersection Crash",  
    "Parking Lot Crash",  
    "Mid Block Crash",  
    "Parked Vehicle Involved",  
]  
  
for col in binary_cols:  
    print(f"\n{col}")  
    print(vancouver_df[col].value_counts())
```

Animal Involved
Animal Involved
No 224915
Yes 233
Name: count, dtype: int64

Cyclist Involved
Cyclist Involved
No 220506
Yes 4642
Name: count, dtype: int64

Heavy Veh Involved
Heavy Veh Involved
No 211156
Yes 13992
Name: count, dtype: int64

Motorcycle Involved
Motorcycle Involved
No 223121
Yes 2027
Name: count, dtype: int64

Pedestrian Involved
Pedestrian Involved
No 221126
Yes 4022
Name: count, dtype: int64

Intersection Crash
Intersection Crash
Yes 131774
No 93374
Name: count, dtype: int64

Parking Lot Crash
Parking Lot Crash
No 174547
Yes 50601
Name: count, dtype: int64

Mid Block Crash
Mid Block Crash
N 202183
Y 22965
Name: count, dtype: int64

Parked Vehicle Involved
Parked Vehicle Involved
No 150995
Yes 74153
Name: count, dtype: int64

Binary variable and Crash Severity

```
In [25]: binary_cols = [
    "Cyclist Involved",
    "Pedestrian Involved",
    "Motorcycle Involved",
    "Heavy Veh Involved",
    "Intersection Crash",
    "Mid Block Crash",
    "Parking Lot Crash",
]

for col in binary_cols:

    print("=" * 60)
    print(col)

    display(pd.crosstab(df[col], df["Crash Severity"], normalize="index").round(3))
```

```
=====
Cyclist Involved
```

Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Cyclist Involved		
No	0.192	0.808
Yes	0.614	0.386

```
=====
Pedestrian Involved
```

Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Pedestrian Involved		
No	0.19	0.81
Yes	0.76	0.24

```
=====
Motorcycle Involved
```

Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Motorcycle Involved		
No	0.194	0.806
Yes	0.556	0.444

```
=====
Heavy Veh Involved
```


Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Heavy Veh Involved		
No	0.198	0.802
Yes	0.158	0.842

=====

Intersection Crash

Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Intersection Crash		
No	0.082	0.918
Yes	0.350	0.650

=====

Mid Block Crash

Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Mid Block Crash		
N	0.217	0.783
Y	0.043	0.957

=====

Parking Lot Crash

Crash Severity	CASUALTY CRASH	PROPERTY DAMAGE ONLY
Parking Lot Crash		
No	0.265	0.735
Yes	0.043	0.957

Vancouver binary variable and casualty heatmap

```
In [26]: binary = [
    "Animal Involved",
    "Cyclist Involved",
    "Heavy Veh Involved",
    "Motorcycle Involved",
    "Pedestrian Involved",
    "Intersection Crash",
    "Parking Lot Crash",
    "Parked Vehicle Involved",
]

binary_df = vancouver_df[binary].replace({"Yes": 1, "No": 0, "Y": 1, "N": 0})

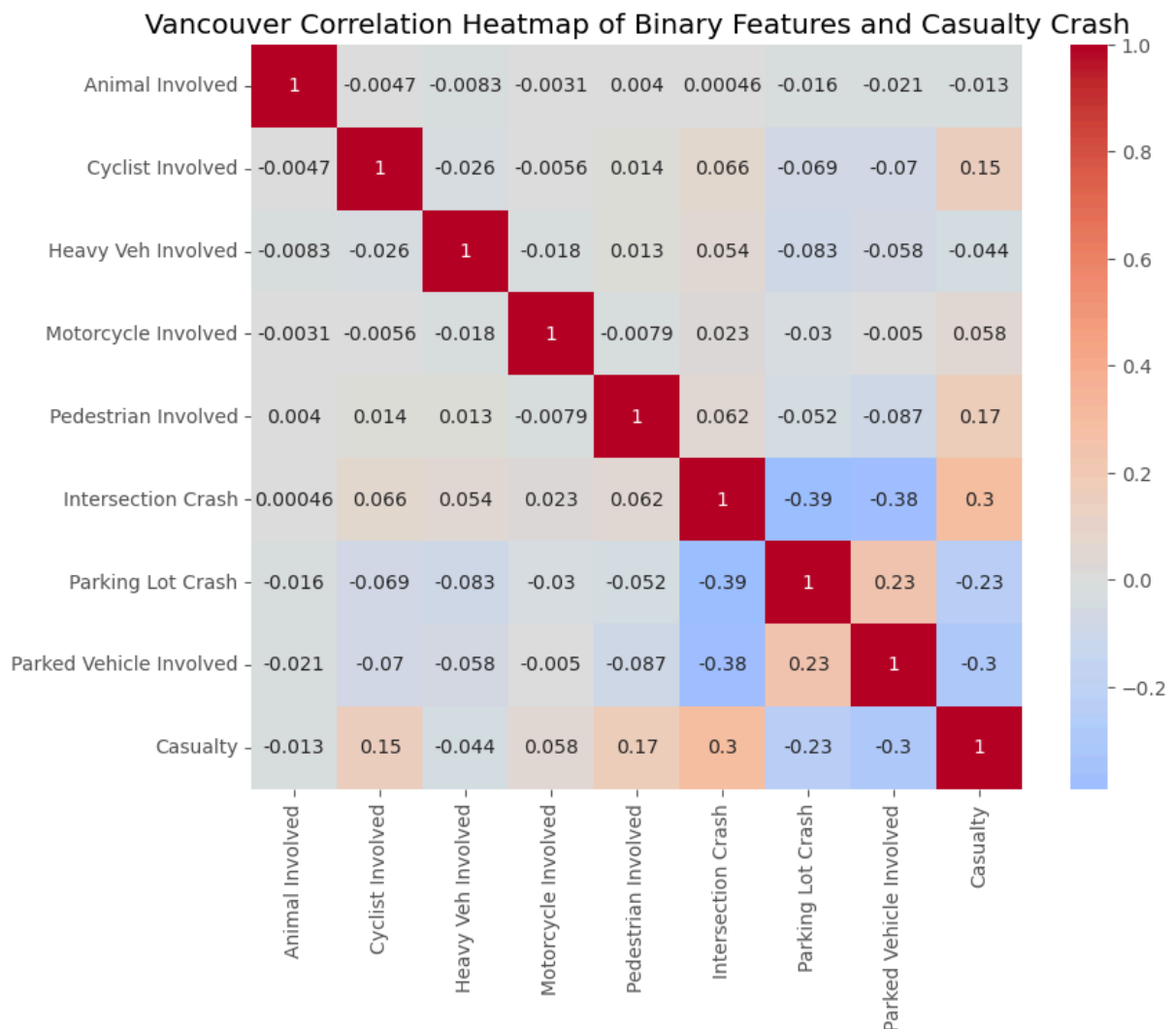
binary_df["Casualty"] = (vancouver_df["Crash Severity"] == "CASUALTY CRASH").astype
```

```
C:\Users\kylez\AppData\Local\Temp\ipykernel_19172\3322405480.py:12: FutureWarning: D
owncasting behavior in `replace` is deprecated and will be removed in a future versi
on. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`.
To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting',
True)`
binary_df = vancouver_df[binary].replace({"Yes": 1, "No": 0, "Y": 1, "N": 0})
```

```
In [27]: plt.figure(figsize=(9, 7))
plt.title("Vancouver Correlation Heatmap of Binary Features and Casualty Crash")

sns.heatmap(binary_df.corr(), annot=True, cmap="coolwarm", center=0)
```

```
Out[27]: <Axes: title={'center': 'Vancouver Correlation Heatmap of Binary Features and Casu
alty Crash'}>
```



Vancouver Intersection data QA

```
In [28]: print(vancouver_df["Intersection Crash"].value_counts())
```

```
Intersection Crash
Yes    131774
No     93374
Name: count, dtype: int64
```

```
In [29]: print(vancouver_df["Cross Street Full Name"].isna().mean())
```

0.3833211931707144

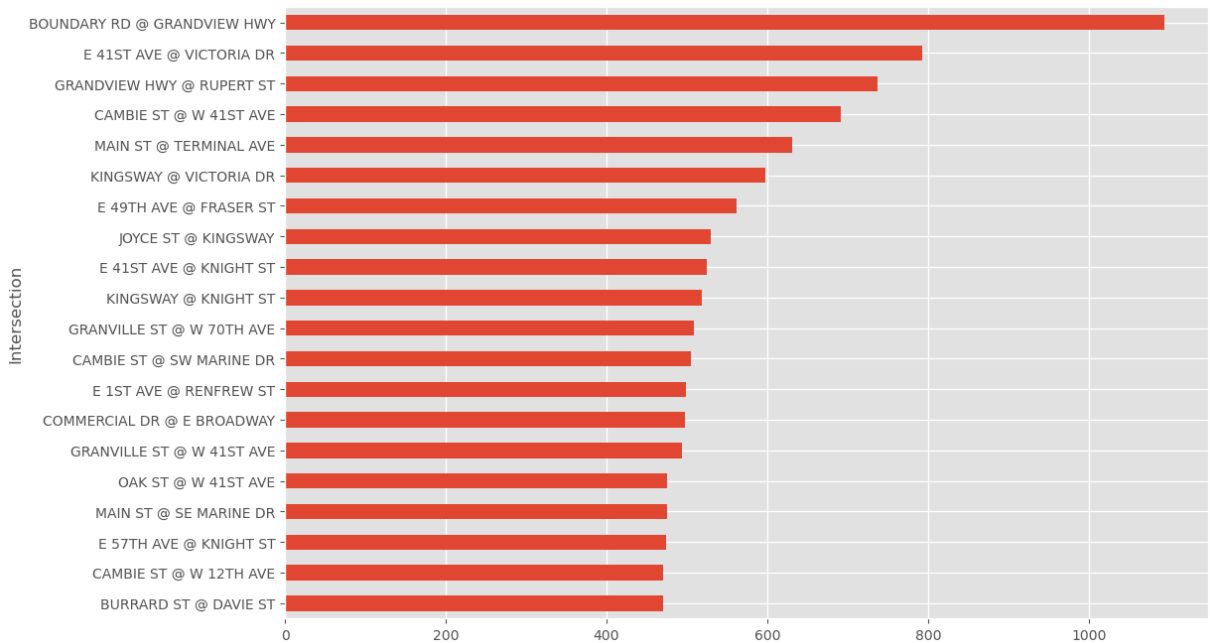
```
In [30]: print(vancouver_df["Street Full Name (ifnull)"].isna().mean())
```

5.3298274912502e-05

```
In [31]: # top 20 Vancouver intersections by total crashes
```

```
intersection_df["Intersection"].value_counts().head(20).sort_values().plot.barh(
    figsize=(12, 8)
)
```

```
Out[31]: <Axes: ylabel='Intersection'>
```



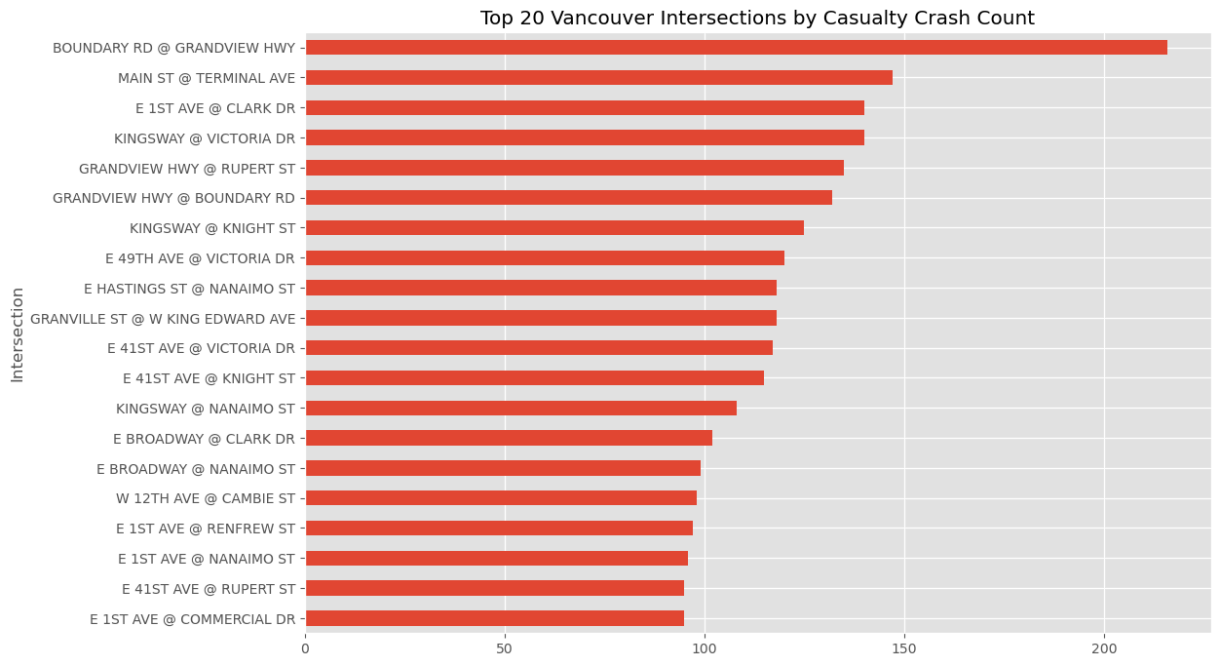
```
In [32]: # Top intersections by casualty crashes in Vancouver
```

```
casualty = vancouver_df[
    (vancouver_df["Crash Severity"] == "CASUALTY CRASH")
    & (vancouver_df["Intersection Crash"] == "Yes")
    & (vancouver_df["Cross Street Full Name"].notna())
].copy()

casualty["Intersection"] = (
    casualty["Street Full Name (ifnull)"] + " @ " + casualty["Cross Street Full Name"]
)

casualty["Intersection"].value_counts().head(20).sort_values().plot.barh(
    figsize=(12, 8), title="Top 20 Vancouver Intersections by Casualty Crash Count"
)
```

```
Out[32]: <Axes: title={'center': 'Top 20 Vancouver Intersections by Casualty Crash Count'},
ylabel='Intersection'>
```

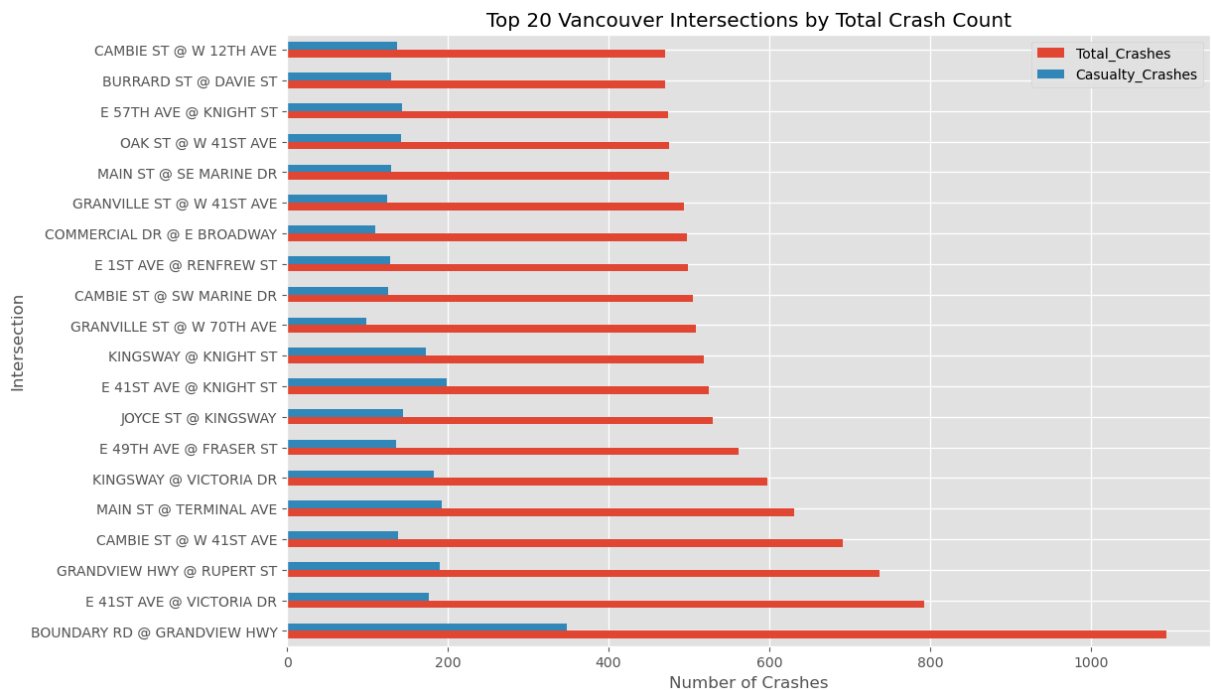


```
In [33]: # summarize crashes by normalized Vancouver intersection
summary = intersection_df.groupby("Intersection").agg(
    Total_Crashes=("Crash Severity", "size"),
    Casualty_Crashes=("Crash Severity", lambda x: (x == "CASUALTY CRASH").sum()),
)

top20 = summary.sort_values("Total_Crashes", ascending=False).head(20)

top20.plot.barh(y=["Total_Crashes", "Casualty_Crashes"], figsize=(12, 8))

plt.title("Top 20 Vancouver Intersections by Total Crash Count")
plt.xlabel("Number of Crashes")
plt.show()
```



Total vs Casualty Crashes by vancouver intersection

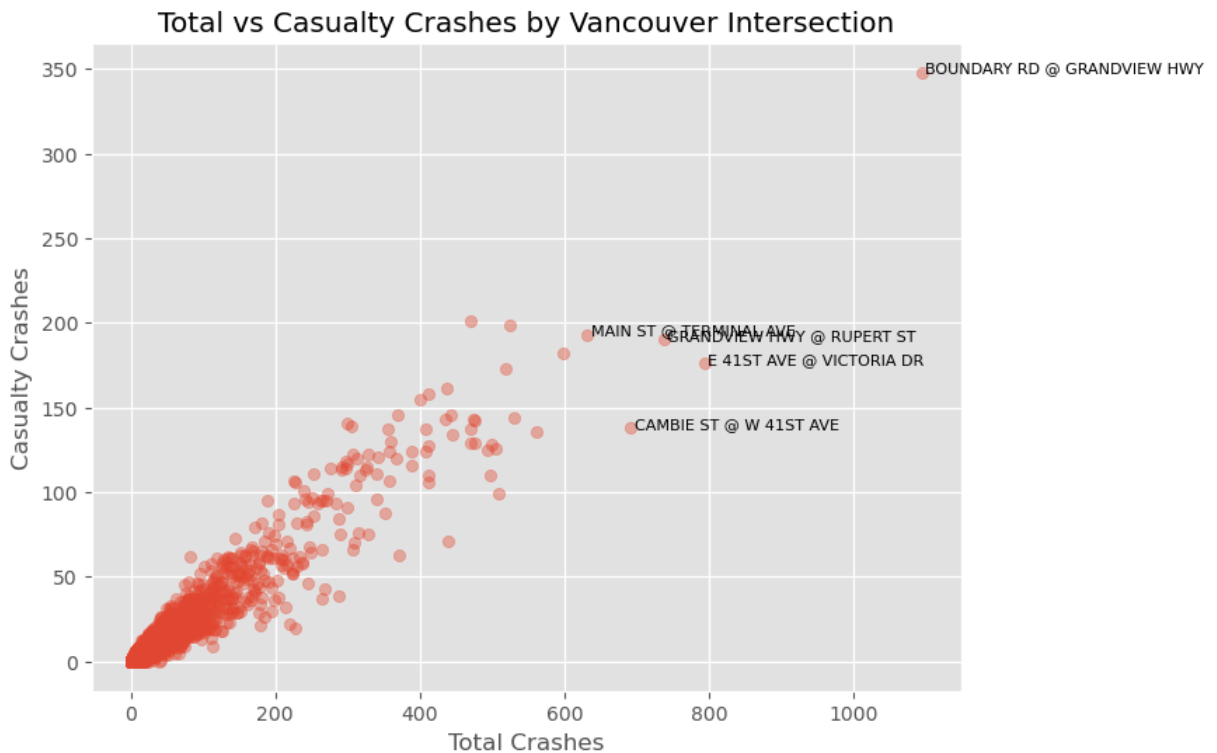
```
In [34]: top = summary.nlargest(5, "Total_Crashes")

plt.figure(figsize=(8, 6))
plt.scatter(summary["Total_Crashes"], summary["Casualty_Crashes"], alpha=0.4)

for idx, row in top.iterrows():
    plt.text(row["Total_Crashes"] + 5, row["Casualty_Crashes"], idx, fontsize=8)

plt.xlabel("Total Crashes")
plt.ylabel("Casualty Crashes")
plt.title("Total vs Casualty Crashes by Vancouver Intersection")
```

Out[34]: Text(0.5, 1.0, 'Total vs Casualty Crashes by Vancouver Intersection')



Total vs Casualty Crashes by Vancouver Intersections Normalized

```
In [35]: # summarize total and casualty crashes by normalized intersection
summary = intersection_df.groupby("Intersection").agg(
    Total_Crashes=("Crash Severity", "size"),
    Casualty_Crashes=("Crash Severity", lambda x: (x == "CASUALTY CRASH").sum()),
)

# label the 5 intersections with the most total crashes
top = summary.nlargest(5, "Total_Crashes")

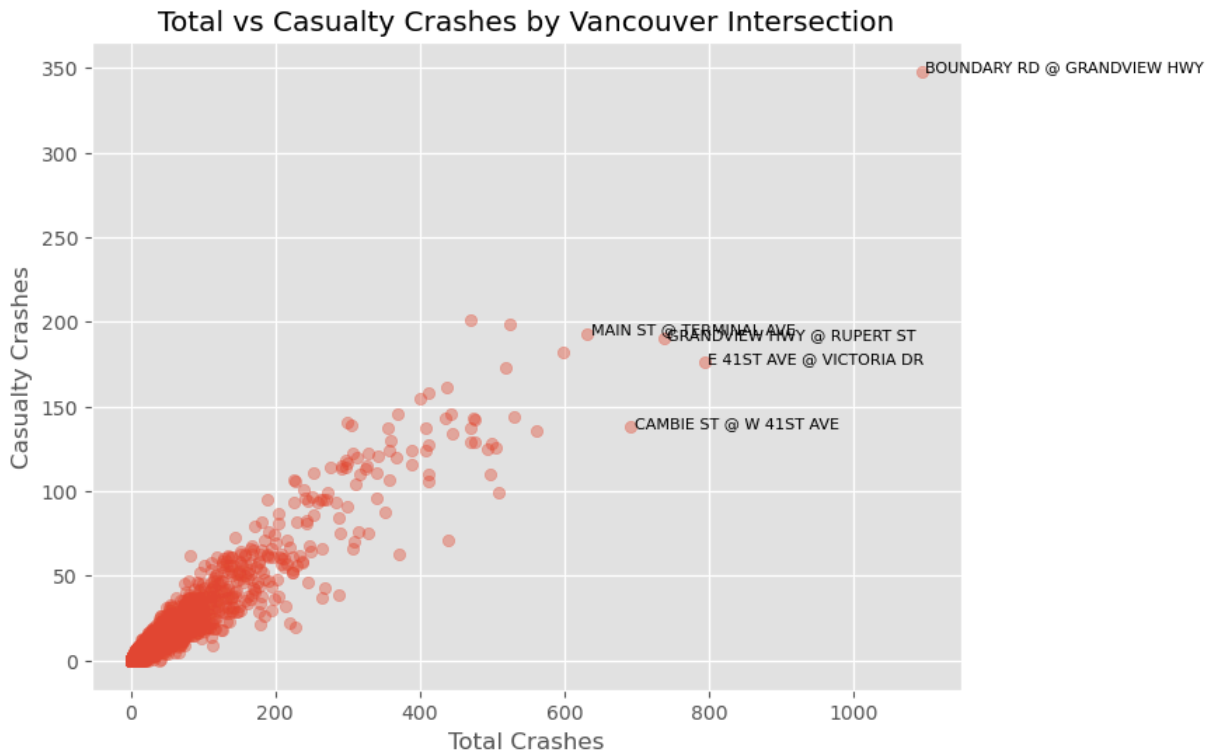
plt.figure(figsize=(8, 6))
```

```
plt.scatter(summary["Total_Crashes"], summary["Casualty_Crashes"], alpha=0.4)

for intersection, row in top.iterrows():
    plt.text(
        row["Total_Crashes"] + 5, row["Casualty_Crashes"], intersection, fontsize=8
    )

plt.xlabel("Total Crashes")
plt.ylabel("Casualty Crashes")
plt.title("Total vs Casualty Crashes by Vancouver Intersection")

plt.show()
```



```
In [36]: corr = summary["Total_Crashes"].corr(summary["Casualty_Crashes"])
print(f"Pearson correlation: {corr:.3f}")
```

Pearson correlation: 0.960

- There is a strong positive relationship between the total number of crashes at an intersection and the number of casualty crashes. Intersections with higher crash volumes also tend to experience more injury crashes.
- Most intersections cluster near the origin, indicating that the majority of Vancouver intersections experience relatively few crashes over the five-year period.
- A small number of intersections stand out as high-volume outliers, with Boundary Rd @ Grandview Hwy being the most prominent.
- There are a few intersections with similar total crash counts but noticeably different casualty counts (for example, W 41st Ave @ Cambie St versus Kingsway @ Victoria Dr), suggesting that crash volume alone may not fully explain casualty risk.

```
In [37]: vancouver_df[["Latitude", "Longitude", "Municipality With Boundary"]].isna().mean()
```

```
Out[37]: Latitude          0.102031  
Longitude          0.102031  
Municipality With Boundary  0.102031  
dtype: float64
```

```
In [38]: (vancouver_df["Latitude"].isna() == vancouver_df["Longitude"].isna()).mean()
```

```
Out[38]: np.float64(1.0)
```

```
In [39]: intersection_df[["Latitude", "Longitude"]].isna().mean()
```

```
Out[39]: Latitude    0.0  
Longitude    0.0  
dtype: float64
```

Within the Vancouver subset, approximately 10.2% of records are missing latitude and longitude coordinates. These fields are always missing together, indicating a common source of missingness rather than independent data quality issues. Since the majority (89.8%) of crash records retain valid coordinates, there is sufficient spatial information for exploratory geographic analyses. Coordinate availability will be considered when selecting records for any map-based visualizations.

1.2 Summary of Findings

The EDA focused on Vancouver crashes from 2018–2022. I looked at data quality, crash severity, temporal patterns, crash characteristics, and differences between intersections.

Key Findings

- The Vancouver subset contains approximately **225,000** crash records between **2018 - 2022**, with roughly **20%** classified as casualty crashes and **80%** as property-damage-only crashes. This represents a moderately imbalanced classification problem.
- Crash frequency varies over time. Most crashes occur during weekday daytime and afternoon hours (approximately **12:00–18:00**), with fewer crashes overnight. A noticeable decline in crash counts is observed during **2020**, followed by recovery in 2021–2022.
- Rear-end, side-impact, and single-vehicle collisions are the most common crash configurations within Vancouver.
- Several binary crash characteristics show meaningful relationships with casualty crashes. Crashes involving pedestrians, cyclists, motorcycles, and intersections are substantially more likely to result in injuries than the overall average, while parking lot crashes are rarely casualty crashes.

- A relatively small number of intersections account for a disproportionately large number of crashes, suggesting that historical intersection crash patterns may be valuable predictors of future casualty risk.
- Missing data is generally limited. The primary missing field is **Cross Street Full Name**, which is expected because many crashes occur away from intersections. This will be considered during feature engineering.

Implications for Modeling

The exploratory analysis suggests that historical crash records contain meaningful information for predicting future casualty crash risk at Vancouver intersections. Variables describing crash history, temporal patterns, crash configuration, and vulnerable road user involvement exhibit meaningful associations with casualty crashes and will be explored as candidate predictive features during model development.

The client's objective is interpreted as forecasting future casualty crash risk to prioritize engineering interventions. Because the dataset ends in 2022, no 2023 observations are available for evaluating a true forward prediction. Therefore, this project uses historical crash information available through 2021 to predict casualty crashes in 2022. This creates a realistic historical forecasting scenario that allows model performance to be quantitatively evaluated. Once validated, the same modeling approach could be retrained using all available data and applied operationally to forecast risk in subsequent years.

The next phase of the project will aggregate individual crash records into intersection-year summaries and engineer historical features for each intersection.

2. Predictive Modeling

Modeling Strategy

The objective of the modeling phase is to identify Vancouver intersections that are most likely to experience casualty crashes in the future so that Engineering Services can prioritize locations for safety review.

The available dataset contains crash records from 2018–2022. To evaluate the forecasting approach using observed data, crashes from 2018–2021 will be used to construct historical features for each intersection, while the number of casualty crashes occurring in 2022 will be used as the prediction target.

Each row in the modeling dataset therefore represents a Vancouver intersection rather than an individual crash.

The primary prediction target is:

2022 casualty crash count per intersection

Historical features will summarize each intersection's previous crash activity, including total crash volume, casualty crash history, crash characteristics, vulnerable road user involvement, and temporal patterns.

Model performance will be evaluated both by its ability to predict casualty crash counts and, more importantly for the client, by how effectively it identifies the highest-risk intersections. Particular attention will therefore be given to the model's top-30 ranked intersections.

2.1 Building the intersection dataset

```
In [40]: # Vancouver intersection crashes only

model_df = vancouver_df[
    (vancouver_df["Intersection Crash"] == "Yes")
    & (vancouver_df["Cross Street Full Name"].notna())
].copy()

model_df["Intersection"] = model_df.apply(normalize_intersection, axis=1)

print(model_df.shape)
print("Unique normalized intersections:", model_df["Intersection"].nunique())

(131774, 30)
Unique normalized intersections: 5617
```

```
In [41]: model_df["Date Of Loss Year"].value_counts().sort_index()
```

```
Out[41]: Date Of Loss Year
2018     32613
2019     31445
2020     19524
2021     22751
2022     25441
Name: count, dtype: int64
```

```
In [42]: # historical data used to predict 2022
train_df = model_df[
    model_df["Date Of Loss Year"] <= 2021
].copy()

# held-out year for evaluation
test_df = model_df[
    model_df["Date Of Loss Year"] == 2022
].copy()

print("Training crashes:", len(train_df))
print("2022 crashes:", len(test_df))
```

```
print("Training intersections:", train_df["Intersection"].nunique())
print("2022 intersections:", test_df["Intersection"].nunique())
```

Training crashes: 106333
 2022 crashes: 25441
 Training intersections: 5337
 2022 intersections: 3676

2.2 Define 2022 Target

```
In [43]: # actual 2022 casualty crashes by intersection

target_2022 = (
    test_df[test_df["Crash Severity"] == "CASUALTY CRASH"]
    .groupby("Intersection")
    .size()
    .rename("Target_Casualty_Crashes_2022")
)

target_2022.head()
```

```
Out[43]: Intersection
ABBOTT ST @ EXPO BLVD      6
ABBOTT ST @ PACIFIC BLVD   1
ABBOTT ST @ W CORDOVA ST   2
ABBOTT ST @ W HASTINGS ST  7
ABBOTT ST @ W PENDER ST    8
Name: Target_Casualty_Crashes_2022, dtype: int64
```

Note: zero casualty crashes in 2022 won't appear in this groupby above.

```
In [44]: # create a dataframe of intersections with target variable for 2022 crashes
intersections = pd.DataFrame({"Intersection": train_df["Intersection"].unique()})

# merge the target variable for 2022 crashes with the intersections dataframe
intersections = intersections.merge(target_2022, on="Intersection", how="left")

# fill missing values with 0 for intersections with no casualty crashes in 2022
intersections["Target_Casualty_Crashes_2022"] = intersections[
    "Target_Casualty_Crashes_2022"
].fillna(0)

intersections.head()
```

```
Out[44]:
```

	Intersection	Target_Casualty_Crashes_2022
0	GRANVILLE ST @ W 70TH AVE	15.0
1	BURRARD ST @ W 4TH AVE	19.0
2	ARBUTUS ST @ W 16TH AVE	8.0
3	CARDERO ST @ DAVIE ST	2.0
4	MILTON ST @ SW MARINE DR	10.0

```
In [45]: intersections.shape
```

```
Out[45]: (5337, 2)
```

```
In [46]: intersections["Target_Casualty_Crashes_2022"].describe()
```

```
Out[46]: count      5337.000000
         mean         1.297358
         std          3.453294
         min          0.000000
         25%          0.000000
         50%          0.000000
         75%          1.000000
         max          58.000000
         Name: Target_Casualty_Crashes_2022, dtype: float64
```

```
In [47]: intersections["Target_Casualty_Crashes_2022"].value_counts().sort_index().head(20)
```

```
Out[47]: Target_Casualty_Crashes_2022
0.0      3474
1.0       789
2.0       339
3.0       182
4.0       145
5.0        84
6.0        58
7.0        44
8.0        34
9.0        28
10.0       25
11.0       14
12.0       13
13.0       15
14.0       17
15.0        4
16.0        7
17.0        8
18.0        7
19.0        6
         Name: count, dtype: int64
```

There are 8,755 intersections, but 6,243 (71.3%) had zero casualty crashes in 2022. The mean is only 0.78 while the maximum is 34, so the target is extremely right-skewed and zero-heavy.

The client mainly cares about whether our model gets the highest-risk ~30 intersections right, not whether it predicts every low-risk intersection perfectly.

So the eventual comparisons could be:

- Baseline: rank by historical crash count alone.
- Random Forest / Gradient Boosting: captures nonlinear relationships between historical characteristics and future casualty counts.

2.3 Feature Engineering

```
In [48]: # Basic crash-history features

# casualty indicator
train_df["Casualty"] = (train_df["Crash Severity"] == "CASUALTY CRASH").astype(int)

# create intersection-level historical features
features = (
    train_df.groupby("Intersection")
    .agg(
        Total_Crashes=("Intersection", "size"),
        Historical_Casualty_Crashes=("Casualty", "sum"),
    )
    .reset_index()
)

features.head()
```

```
Out[48]:
```

	Intersection	Total_Crashes	Historical_Casualty_Crashes
0	29TH AVE STATION LOOP @ E 29TH AVE	1	0
1	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	4	2
2	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	4	1
3	ABBOTT ST @ EXPO BLVD	68	22
4	ABBOTT ST @ KEEFER PL	65	9

```
In [49]: # create a new feature for historical casualty crash rate
features["Historical_Casualty_Rate"] = (
    features["Historical_Casualty_Crashes"] / features["Total_Crashes"]
)
```

```
In [50]: # converting variables we found during EDA to binary features for modeling

train_df["Pedestrian"] = (train_df["Pedestrian Involved"] == "Yes").astype(int)
train_df["Cyclist"] = (train_df["Cyclist Involved"] == "Yes").astype(int)
train_df["Motorcycle"] = (train_df["Motorcycle Involved"] == "Yes").astype(int)
```

```
train_df["Heavy_Vehicle"] = (train_df["Heavy Veh Involved"] == "Yes").astype(int)
train_df["Parked_Vehicle"] = (train_df["Parked Vehicle Involved"] == "Yes").astype(int)
```

```
In [51]: # aggregate the binary features to intersection level
characteristics = (
    train_df.groupby("Intersection")
    .agg(
        Pedestrian_Crashes=("Pedestrian", "sum"),
        Cyclist_Crashes=("Cyclist", "sum"),
        Motorcycle_Crashes=("Motorcycle", "sum"),
        Heavy_Vehicle_Crashes=("Heavy_Vehicle", "sum"),
        Parked_Vehicle_Crashes=("Parked_Vehicle", "sum"),
    )
    .reset_index()
)
```

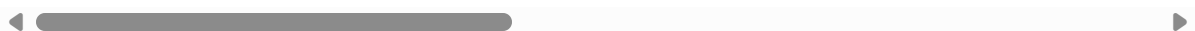
```
In [52]: # merge the characteristics dataframe with the features dataframe
features = features.merge(characteristics, on="Intersection", how="left")
```

```
In [53]: model_data = features.merge(intersections, on="Intersection", how="left")

model_data.head()
```

```
Out[53]:
```

	Intersection	Total_Crashes	Historical_Casualty_Crashes	Historical_Casualty_Rate	Pedestri
0	29TH AVE STATION LOOP @ E 29TH AVE	1	0	0.000000	
1	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	4	2	0.500000	
2	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	4	1	0.250000	
3	ABBOTT ST @ EXPO BLVD	68	22	0.323529	
4	ABBOTT ST @ KEEFER PL	65	9	0.138462	



```
In [54]: # select the relevant columns for modeling
```

```

model_data = model_data[
    [
        "Intersection",
        "Total_Crashes",
        "Historical_Casualty_Crashes",
        "Historical_Casualty_Rate",
        "Pedestrian_Crashes",
        "Cyclist_Crashes",
        "Motorcycle_Crashes",
        "Heavy_Vehicle_Crashes",
        "Parked_Vehicle_Crashes",
        "Target_Casualty_Crashes_2022",
    ]
].copy()

model_data.head()

```

Out[54]:

	Intersection	Total_Crashes	Historical_Casualty_Crashes	Historical_Casualty_Rate	Pedestri
--	--------------	---------------	-----------------------------	--------------------------	----------

0	29TH AVE STATION LOOP @ E 29TH AVE	1	0	0.000000	
1	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	4	2	0.500000	
2	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	4	1	0.250000	
3	ABBOTT ST @ EXPO BLVD	68	22	0.323529	
4	ABBOTT ST @ KEEFER PL	65	9	0.138462	



In [55]: `model_data.shape`

Out[55]: (5337, 10)

In [56]: `# annual crash counts`

```

yearly_crashes = (
    train_df.groupby(["Intersection", "Date Of Loss Year"]).size().unstack(fill_val
)

```

```
yearly_crashes.columns = [f"Crashes_{int(year)}" for year in yearly_crashes.columns]
yearly_crashes = yearly_crashes.reset_index()
yearly_crashes.head()
```

Out[56]:

	Intersection	Crashes_2018	Crashes_2019	Crashes_2020	Crashes_2021
0	29TH AVE STATION LOOP @ E 29TH AVE	0	0	0	1
1	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	1	1	1	1
2	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	3	0	1	0
3	ABBOTT ST @ EXPO BLVD	25	21	10	12
4	ABBOTT ST @ KEEFER PL	21	18	10	16

In [57]: *# annual casualty crash counts*

```
yearly_casualties = (
    train_df.groupby(["Intersection", "Date Of Loss Year"])["Casualty"]
    .sum()
    .unstack(fill_value=0)
)

yearly_casualties.columns = [
    f"Casualty_Crashes_{int(year)}" for year in yearly_casualties.columns
]

yearly_casualties = yearly_casualties.reset_index()
yearly_casualties.head()
```

Out[57]:

	Intersection	Casualty_Crashes_2018	Casualty_Crashes_2019	Casualty_Crashes_2020	Casu
--	--------------	-----------------------	-----------------------	-----------------------	------

0	29TH AVE STATION LOOP @ E 29TH AVE	0	0	0
1	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	1	1	0
2	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	1	0	0
3	ABBOTT ST @ EXPO BLVD	7	9	3
4	ABBOTT ST @ KEEFER PL	2	4	1



In [58]: *# merge the annual crash counts and casualty crash counts with the model_data dataframe*

```
model_data = model_data.merge(yearly_crashes, on="Intersection", how="left").merge(
    yearly_casualties, on="Intersection", how="left"
)

model_data.head()
```


Out[58]:

	Intersection	Total_Crashes	Historical_Casualty_Crashes	Historical_Casualty_Rate	Pedestri
--	--------------	---------------	-----------------------------	--------------------------	----------

0	29TH AVE STATION LOOP @ E 29TH AVE	1	0	0.000000	
---	---	---	---	----------	--

1	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	4	2	0.500000	
---	---	---	---	----------	--

2	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	4	1	0.250000	
---	---	---	---	----------	--

3	ABBOTT ST @ EXPO BLVD	68	22	0.323529	
---	-----------------------------	----	----	----------	--

4	ABBOTT ST @ KEEFER PL	65	9	0.138462	
---	-----------------------------	----	---	----------	--



In [59]: `model_data.shape`

Out[59]: (5337, 18)

```
In [60]: # assert that the sum of annual crash counts equals the total crashes
assert (
    model_data[["Crashes_2018", "Crashes_2019", "Crashes_2020", "Crashes_2021"]].sum(
        axis=1
    )
    == model_data["Total_Crashes"]
).all()

# assert that the sum of annual casualty crash counts equals the historical casualty
assert (
    model_data[
        [
            "Casualty_Crashes_2018",
            "Casualty_Crashes_2019",
            "Casualty_Crashes_2020",
            "Casualty_Crashes_2021",
        ]
    ].sum(axis=1)
    == model_data["Historical_Casualty_Crashes"]
).all()
```

```
In [61]: # assert that intersection names are normalized consistently
for intersection in model_data["Intersection"]:
    street1, street2 = intersection.split(" @ ", 1)
    assert street1 <= street2

print("All intersection names are normalized.")
```

All intersection names are normalized.

2.4 Baseline

```
In [62]: # Baseline model evaluation

y_true = model_data["Target_Casualty_Crashes_2022"]

baseline_pred = model_data["Casualty_Crashes_2021"]

# calculate baseline MAE and RMSE
baseline_mae = mean_absolute_error(y_true, baseline_pred)
baseline_rmse = np.sqrt(mean_squared_error(y_true, baseline_pred))

print(f"Baseline MAE: {baseline_mae:.3f}")
print(f"Baseline RMSE: {baseline_rmse:.3f}")
```

Baseline MAE: 0.812

Baseline RMSE: 1.748

Since the client cares about the top 30:

```
In [63]: # overlap of top-30 intersections by casualty crashes in 2021 and 2022
actual_top30 = set(
    model_data.nlargest(30, "Target_Casualty_Crashes_2022")["Intersection"]
)

# nlargest returns a DataFrame, so we need to select the "Intersection" column and
baseline_top30 = set(model_data.nlargest(30, "Casualty_Crashes_2021")["Intersection"])

baseline_overlap = len(actual_top30 & baseline_top30)

print(f"Baseline Top-30 overlap:{baseline_overlap}/30")
```

Baseline Top-30 overlap:16/30

Note: nlargest arbitrarily takes the top 30.

A simple historical baseline was established by assuming that each intersection's casualty crash count in 2022 would equal its casualty crash count in 2021.

The baseline achieved:

- **MAE:** 0.624 casualty crashes
- **RMSE:** 1.366 casualty crashes
- **Top-30 overlap:** 14/30 intersections (46.7%)

Although the average prediction error is relatively small, this partly reflects the large number of intersections with few or no casualty crashes. More importantly for the stakeholder, using the previous year's casualty count alone identifies fewer than half of the 30 highest-casualty intersections in 2022.

The predictive models will therefore be evaluated not only on overall prediction error, but also on whether they improve identification of the highest-risk intersections relative to this historical baseline.

2.5 Historical forecasting samples

```
In [64]: def build_forecast_dataset(df, target_year):
    history = df[df["Date Of Loss Year"] < target_year].copy()
    future = df[df["Date Of Loss Year"] == target_year].copy()

    # binary indicators
    history["Casualty"] = (
        history["Crash Severity"] == "CASUALTY CRASH"
    ).astype(int)

    history["Pedestrian"] = (
        history["Pedestrian Involved"] == "Yes"
    ).astype(int)

    history["Cyclist"] = (
        history["Cyclist Involved"] == "Yes"
    ).astype(int)

    history["Motorcycle"] = (
        history["Motorcycle Involved"] == "Yes"
    ).astype(int)

    history["Heavy_Vehicle"] = (
        history["Heavy Veh Involved"] == "Yes"
    ).astype(int)

    history["Parked_Vehicle"] = (
        history["Parked Vehicle Involved"] == "Yes"
    ).astype(int)

    # cumulative historical features
    features = history.groupby("Intersection").agg(
        Total_Crashes=("Intersection", "size"),
        Historical_Casualty_Crashes=("Casualty", "sum"),
        Pedestrian_Crashes=("Pedestrian", "sum"),
        Cyclist_Crashes=("Cyclist", "sum"),
        Motorcycle_Crashes=("Motorcycle", "sum"),
        Heavy_Vehicle_Crashes=("Heavy_Vehicle", "sum"),
        Parked_Vehicle_Crashes=("Parked_Vehicle", "sum"),
    ).reset_index()

    features["Historical_Casualty_Rate"] = (
        features["Historical_Casualty_Crashes"]
```

```

        / features["Total_Crashes"]
    )

    # temporal features: previous year and two years ago crash counts and casualty
    previous_year = target_year - 1
    two_years_ago = target_year - 2

    def get_year_counts(year):
        year_df = history[
            history["Date Of Loss Year"] == year
        ]

        crashes = year_df.groupby("Intersection").size()

        casualties = (
            year_df[
                year_df["Crash Severity"] == "CASUALTY CRASH"
            ]
            .groupby("Intersection")
            .size()
        )

        return crashes, casualties

    prev_crashes, prev_casualties = get_year_counts(previous_year)
    older_crashes, older_casualties = get_year_counts(two_years_ago)

    features["Previous_Year_Crashes"] = (
        features["Intersection"].map(prev_crashes).fillna(0)
    )

    features["Previous_Year_Casualty_Crashes"] = (
        features["Intersection"].map(prev_casualties).fillna(0)
    )

    features["Two_Years_Ago_Crashes"] = (
        features["Intersection"].map(older_crashes).fillna(0)
    )

    features["Two_Years_Ago_Casualty_Crashes"] = (
        features["Intersection"].map(older_casualties).fillna(0)
    )

    # casualty rate in most recent year
    features["Previous_Year_Casualty_Rate"] = np.where(
        features["Previous_Year_Crashes"] > 0,
        features["Previous_Year_Casualty_Crashes"]
        / features["Previous_Year_Crashes"],
        0
    )

    # average of previous two years
    features["Two_Year_Avg_Crashes"] = (
        features["Previous_Year_Crashes"]
        + features["Two_Years_Ago_Crashes"]
    ) / 2

```

```

features["Two_Year_Avg_Casualties"] = (
    features["Previous_Year_Casualty_Crashes"]
    + features["Two_Years_Ago_Casualty_Crashes"]
) / 2

# change from two years ago to previous year
features["Crash_Trend"] = (
    features["Previous_Year_Crashes"]
    - features["Two_Years_Ago_Crashes"]
)

features["Casualty_Trend"] = (
    features["Previous_Year_Casualty_Crashes"]
    - features["Two_Years_Ago_Casualty_Crashes"]
)

target = (
    future[
        future["Crash Severity"] == "CASUALTY CRASH"
    ]
    .groupby("Intersection")
    .size()
    .rename("Target")
)

features = features.merge(
    target,
    on="Intersection",
    how="left"
)

features["Target"] = features["Target"].fillna(0)
features["Target_Year"] = target_year

return features

```

```

In [65]: forecast_2020 = build_forecast_dataset(model_df, 2020)
forecast_2021 = build_forecast_dataset(model_df, 2021)
forecast_2022 = build_forecast_dataset(model_df, 2022)

print("2020:", forecast_2020.shape)
print("2021:", forecast_2021.shape)
print("2022:", forecast_2022.shape)

```

```

2020: (4625, 20)
2021: (5006, 20)
2022: (5337, 20)

```

```

In [66]: forecast_2020.head()

```

Out[66]:

	Intersection	Total_Crashes	Historical_Casualty_Crashes	Pedestrian_Crashes	Cyclist_Crashes
--	--------------	---------------	-----------------------------	--------------------	-----------------

0	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	2	2	0
---	---	---	---	---

1	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	3	1	0
---	---	---	---	---

2	ABBOTT ST @ EXPO BLVD	46	16	1
---	-----------------------------	----	----	---

3	ABBOTT ST @ KEEFER PL	39	6	2
---	-----------------------------	----	---	---

4	ABBOTT ST @ PACIFIC BLVD	16	5	1
---	--------------------------------	----	---	---



```
In [67]: # training data is 2020 and 2021, test data is 2022

training_data = pd.concat([forecast_2020, forecast_2021], ignore_index=True)

test_data = forecast_2022.copy()

print("Training samples:", len(training_data))
print("2022 test samples:", len(test_data))
```

Training samples: 9631
2022 test samples: 5337

```
In [68]: display(training_data.head())

print("\nTraining target:")
display(training_data["Target"].describe())

print("\n2022 target:")
display(test_data["Target"].describe())

print("\nTarget years:")
print(training_data["Target_Year"].value_counts())
```

	Intersection	Total_Crashes	Historical_Casualty_Crashes	Pedestrian_Crashes	Cyclist_Crashes
0	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	2	2	0	0
1	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	3	1	0	0
2	ABBOTT ST @ EXPO BLVD	46	16	1	3
3	ABBOTT ST @ KEEFER PL	39	6	2	3
4	ABBOTT ST @ PACIFIC BLVD	16	5	1	3



```

Training target:
count    9631.000000
mean      1.339529
std       3.418169
min       0.000000
25%      0.000000
50%      0.000000
75%      1.000000
max      62.000000
Name: Target, dtype: float64
2022 target:
count    5337.000000
mean      1.297358
std       3.453294
min       0.000000
25%      0.000000
50%      0.000000
75%      1.000000
max      58.000000
Name: Target, dtype: float64
Target years:
Target_Year
2021    5006
2020    4625
Name: count, dtype: int64

```

```
In [69]: print(training_data.isna().sum())
```

Intersection	0
Total_Crashes	0
Historical_Casualty_Crashes	0
Pedestrian_Crashes	0
Cyclist_Crashes	0
Motorcycle_Crashes	0
Heavy_Vehicle_Crashes	0
Parked_Vehicle_Crashes	0
Historical_Casualty_Rate	0
Previous_Year_Crashes	0
Previous_Year_Casualty_Crashes	0
Two_Years_Ago_Crashes	0
Two_Years_Ago_Casualty_Crashes	0
Previous_Year_Casualty_Rate	0
Two_Year_Avg_Crashes	0
Two_Year_Avg_Casualties	0
Crash_Trend	0
Casualty_Trend	0
Target	0
Target_Year	0

dtype: int64

- 2020 forecast: 7,244 intersections
- 2021 forecast: 8,025 intersections
- Training: 15,269 intersection-year observations
- Final 2022 test: 8,755 intersections
- Missing values: 0
- 2022 is completely excluded from training targets

2.6 Random Forest

2.6.1 Random Forest Regressor - Cumulative Historical Features

```
In [70]: from sklearn.ensemble import RandomForestRegressor
```

```
In [71]: feature_cols = [
    "Total_Crashes",
    "Historical_Casualty_Crashes",
    "Historical_Casualty_Rate",
    "Pedestrian_Crashes",
    "Cyclist_Crashes",
    "Motorcycle_Crashes",
    "Heavy_Vehicle_Crashes",
    "Parked_Vehicle_Crashes",
    "Previous_Year_Crashes",
    "Previous_Year_Casualty_Crashes",

    # temporal features
    "Two_Years_Ago_Crashes",
```



```

    "Two_Years_Ago_Casualty_Crashes",
    "Previous_Year_Casualty_Rate",
    "Two_Year_Avg_Crashes",
    "Two_Year_Avg_Casualties",
    "Crash_Trend",
    "Casualty_Trend",
]

X_train = training_data[feature_cols]
y_train = training_data["Target"]

X_test = test_data[feature_cols]
y_test = test_data["Target"]

# train a random forest regressor
rf = RandomForestRegressor(
    n_estimators=300, random_state=42, n_jobs=-1, min_samples_leaf=2
)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

# evaluate the model
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))

print(f"Random Forest MAE: {rf_mae:.3f}")
print(f"Random Forest RMSE: {rf_rmse:.3f}")

```

Random Forest MAE: 0.808

Random Forest RMSE: 1.643

```

In [72]: # adding predictions to the 2022 test data for comparison with actual values
results = test_data[["Intersection", "Target"]].copy()

results["Predicted"] = rf_pred

```

```

In [73]: # evaluate the overlap of top-30 intersections by casualty crashes in 2022 and pred

actual_top30 = set(results.nlargest(30, "Target")["Intersection"])

predicted_top30 = set(results.nlargest(30, "Predicted")["Intersection"])

rf_top30_overlap = len(actual_top30 & predicted_top30)

print(
    f"Random Forest Top-30 overlap: "
    f"{rf_top30_overlap}/30 "
    f"({rf_top30_overlap / 30:.1%})"
)

```

Random Forest Top-30 overlap: 17/30 (56.7%)

```

In [74]: # baseline comparison
print("Baseline")
print(f"MAE: {baseline_mae:.3f}")

```

```

print(f"RMSE:          {baseline_rmse:.3f}")
print(f"Top-30:        {baseline_overlap}/30")

print("\nRandom Forest")
print(f"MAE:            {rf_mae:.3f}")
print(f"RMSE:            {rf_rmse:.3f}")
print(f"Top-30:          {rf_top30_overlap}/30")

```

Baseline

```

MAE:          0.812
RMSE:         1.748
Top-30:       16/30

```

Random Forest

```

MAE:          0.808
RMSE:         1.643
Top-30:       17/30

```

Random Forest is not clearly better overall. It slightly improves RMSE and finds one additional intersection in the actual Top 30, but its MAE is slightly worse.

```
In [75]: results.nlargest(30, "Predicted")[["Intersection", "Predicted", "Target"]]
```

Out[75]:

	Intersection	Predicted	Target
672	BOUNDARY RD @ GRANDVIEW HWY	42.728139	58.0
1249	CLARK DR @ E 1ST AVE	37.923258	33.0
4240	MAIN ST @ TERMINAL AVE	35.225020	32.0
2538	E 41ST AVE @ KNIGHT ST	34.687079	36.0
2558	E 41ST AVE @ VICTORIA DR	34.629980	31.0
4015	KINGSWAY @ VICTORIA DR	34.300845	27.0
3495	GRANDVIEW HWY @ RUPERT ST	34.246274	37.0
4039	KNIGHT ST @ SE MARINE DR	33.748163	27.0
3989	KINGSWAY @ KNIGHT ST	33.045437	37.0
2707	E 49TH AVE @ KNIGHT ST	32.683004	18.0
3619	GRANVILLE ST @ W KING EDWARD AVE	32.329409	25.0
1636	DENMAN ST @ W GEORGIA ST	32.198663	17.0
2726	E 49TH AVE @ VICTORIA DR	31.850369	25.0
3167	E HASTINGS ST @ NANAIMO ST	31.584278	18.0
776	BURRARD ST @ DAVIE ST	31.540063	9.0
3569	GRANVILLE ST @ W 16TH AVE	31.515758	24.0
2881	E 57TH AVE @ KNIGHT ST	31.422484	25.0
3997	KINGSWAY @ NANAIMO ST	31.124984	20.0
3583	GRANVILLE ST @ W 41ST AVE	30.001071	13.0
1262	CLARK DR @ E BROADWAY	29.695270	27.0
2076	E 1ST AVE @ RENFREW ST	29.616468	18.0
2700	E 49TH AVE @ FRASER ST	29.253282	16.0
4572	OAK ST @ W 41ST AVE	29.248746	22.0
3413	FRASER ST @ SE MARINE DR	28.115298	13.0
2532	E 41ST AVE @ FRASER ST	27.147980	30.0
869	CAMBIE ST @ W 12TH AVE	27.074032	26.0
866	CAMBIE ST @ SW MARINE DR	26.986329	13.0
2550	E 41ST AVE @ RUPERT ST	26.684357	31.0
4238	MAIN ST @ SE MARINE DR	26.567835	20.0
899	CAMBIE ST @ W 41ST AVE	26.041992	22.0

```

In [76]: rf_results = results.copy()

# actual top 30 intersections in 2022
actual_top30_df = rf_results.nlargest(30, "Target")[
    ["Intersection", "Target", "Predicted"]
].copy()

# predicted top 30 intersections
predicted_top30_df = rf_results.nlargest(30, "Predicted")[
    ["Intersection", "Target", "Predicted"]
].copy()

actual_top30_set = set(actual_top30_df["Intersection"])
predicted_top30_set = set(predicted_top30_df["Intersection"])

correct_top30 = actual_top30_set & predicted_top30_set
missed_top30 = actual_top30_set - predicted_top30_set
false_positive_top30 = predicted_top30_set - actual_top30_set

print(f"Correctly identified: {len(correct_top30)}")
print(f"Missed actual top-30: {len(missed_top30)}")
print(f"Incorrectly promoted into top-30: {len(false_positive_top30)}")

```

Correctly identified: 17
Missed actual top-30: 13
Incorrectly promoted into top-30: 13

```

In [77]: correct_df = rf_results[rf_results["Intersection"].isin(correct_top30)].sort_values(
    "Target", ascending=False
)

missed_df = rf_results[rf_results["Intersection"].isin(missed_top30)].sort_values(
    "Target", ascending=False
)

false_positive_df = rf_results[
    rf_results["Intersection"].isin(false_positive_top30)
].sort_values("Predicted", ascending=False)

display(correct_df[["Intersection", "Target", "Predicted"]])
display(missed_df[["Intersection", "Target", "Predicted"]])
display(false_positive_df[["Intersection", "Target", "Predicted"]])

```

	Intersection	Target	Predicted
672	BOUNDARY RD @ GRANDVIEW HWY	58.0	42.728139
3989	KINGSWAY @ KNIGHT ST	37.0	33.045437
3495	GRANDVIEW HWY @ RUPERT ST	37.0	34.246274
2538	E 41ST AVE @ KNIGHT ST	36.0	34.687079
1249	CLARK DR @ E 1ST AVE	33.0	37.923258
4240	MAIN ST @ TERMINAL AVE	32.0	35.225020
2550	E 41ST AVE @ RUPERT ST	31.0	26.684357
2558	E 41ST AVE @ VICTORIA DR	31.0	34.629980
2532	E 41ST AVE @ FRASER ST	30.0	27.147980
1262	CLARK DR @ E BROADWAY	27.0	29.695270
4015	KINGSWAY @ VICTORIA DR	27.0	34.300845
4039	KNIGHT ST @ SE MARINE DR	27.0	33.748163
869	CAMBIE ST @ W 12TH AVE	26.0	27.074032
2881	E 57TH AVE @ KNIGHT ST	25.0	31.422484
3619	GRANVILLE ST @ W KING EDWARD AVE	25.0	32.329409
2726	E 49TH AVE @ VICTORIA DR	25.0	31.850369
3569	GRANVILLE ST @ W 16TH AVE	24.0	31.515758

	Intersection	Target	Predicted
3945	JOYCE ST @ KINGSWAY	39.0	26.010333
678	BOUNDARY RD @ MARINE WAY	37.0	21.082130
3980	KERR ST @ SE MARINE DR	33.0	25.831400
3098	E BROADWAY @ NANAIMO ST	29.0	23.856446
3106	E BROADWAY @ RUPERT ST	27.0	19.007357
3494	GRANDVIEW HWY @ RENFREW ST	26.0	19.839131
2341	E 2ND AVE @ MAIN ST	25.0	21.612101
4579	OAK ST @ W 49TH AVE	25.0	19.908896
2406	E 33RD AVE @ KNIGHT ST	24.0	21.694272
3589	GRANVILLE ST @ W 49TH AVE	24.0	24.566036
1089	CASSIAR CONNECTR @ E HASTINGS ST	23.0	21.894984
1241	CLARK DR @ E 12TH AVE	23.0	23.046931
1880	E 12TH AVE @ MAIN ST	23.0	21.274915

	Intersection	Target	Predicted
2707	E 49TH AVE @ KNIGHT ST	18.0	32.683004
1636	DENMAN ST @ W GEORGIA ST	17.0	32.198663
3167	E HASTINGS ST @ NANAIMO ST	18.0	31.584278
776	BURRARD ST @ DAVIE ST	9.0	31.540063
3997	KINGSWAY @ NANAIMO ST	20.0	31.124984
3583	GRANVILLE ST @ W 41ST AVE	13.0	30.001071
2076	E 1ST AVE @ RENFREW ST	18.0	29.616468
2700	E 49TH AVE @ FRASER ST	16.0	29.253282
4572	OAK ST @ W 41ST AVE	22.0	29.248746
3413	FRASER ST @ SE MARINE DR	13.0	28.115298
866	CAMBIE ST @ SW MARINE DR	13.0	26.986329
4238	MAIN ST @ SE MARINE DR	20.0	26.567835
899	CAMBIE ST @ W 41ST AVE	22.0	26.041992

Among the misses:

- JOYCE ST @ KINGSWAY: predicted 26.0, actual 39
- BOUNDARY RD @ MARINE WAY: 21.1 → 37

- KERR ST @ SE MARINE DR: 25.8 → 33
- E BROADWAY @ RUPERT ST: 19.0 → 27

Meanwhile some false positives collapsed in 2022:

- BURRARD ST @ DAVIE ST: predicted 31.5, actual 9
- GRANVILLE ST @ W 41ST AVE: 30.0 → 13
- DENMAN ST @ W GEORGIA ST: 32.2 → 17
- E 49TH AVE @ KNIGHT ST: 32.7 → 18

```
In [78]: # add ranks and rank errors to the results dataframe

rf_results["Actual_Rank"] = rf_results["Target"].rank(method="min", ascending=False)

rf_results["Predicted_Rank"] = rf_results["Predicted"].rank(
    method="min", ascending=False
)

rf_results["Rank_Error"] = (
    rf_results["Predicted_Rank"] - rf_results["Actual_Rank"]
).abs()
```

```
In [79]: actual_top30_ranks = rf_results[rf_results["Actual_Rank"] <= 30][
    [
        "Intersection",
        "Target",
        "Predicted",
        "Actual_Rank",
        "Predicted_Rank",
        "Rank_Error",
    ]
].sort_values("Actual_Rank")

display(actual_top30_ranks)
```

	Intersection	Target	Predicted	Actual_Rank	Predicted_Rank	Rank_Error
672	BOUNDARY RD @ GRANDVIEW HWY	58.0	42.728139	1.0	1.0	0.0
3945	JOYCE ST @ KINGSWAY	39.0	26.010333	2.0	31.0	29.0
3989	KINGSWAY @ KNIGHT ST	37.0	33.045437	3.0	9.0	6.0
678	BOUNDARY RD @ MARINE WAY	37.0	21.082130	3.0	53.0	50.0
3495	GRANDVIEW HWY @ RUPERT ST	37.0	34.246274	3.0	7.0	4.0
2538	E 41ST AVE @ KNIGHT ST	36.0	34.687079	6.0	4.0	2.0
1249	CLARK DR @ E 1ST AVE	33.0	37.923258	7.0	2.0	5.0
3980	KERR ST @ SE MARINE DR	33.0	25.831400	7.0	32.0	25.0
4240	MAIN ST @ TERMINAL AVE	32.0	35.225020	9.0	3.0	6.0
2550	E 41ST AVE @ RUPERT ST	31.0	26.684357	10.0	28.0	18.0
2558	E 41ST AVE @ VICTORIA DR	31.0	34.629980	10.0	5.0	5.0
2532	E 41ST AVE @ FRASER ST	30.0	27.147980	12.0	25.0	13.0
3098	E BROADWAY @ NANAIMO ST	29.0	23.856446	13.0	38.0	25.0
4039	KNIGHT ST @ SE MARINE DR	27.0	33.748163	14.0	8.0	6.0
4015	KINGSWAY @ VICTORIA DR	27.0	34.300845	14.0	6.0	8.0
1262	CLARK DR @ E BROADWAY	27.0	29.695270	14.0	20.0	6.0
3106	E BROADWAY @ RUPERT ST	27.0	19.007357	14.0	74.0	60.0
869	CAMBIE ST @ W 12TH AVE	26.0	27.074032	18.0	26.0	8.0
3494	GRANDVIEW HWY @ RENFREW ST	26.0	19.839131	18.0	68.0	50.0
3619	GRANVILLE ST @ W KING EDWARD AVE	25.0	32.329409	20.0	11.0	9.0

	Intersection	Target	Predicted	Actual_Rank	Predicted_Rank	Rank_Error
4579	OAK ST @ W 49TH AVE	25.0	19.908896	20.0	66.0	46.0
2726	E 49TH AVE @ VICTORIA DR	25.0	31.850369	20.0	13.0	7.0
2341	E 2ND AVE @ MAIN ST	25.0	21.612101	20.0	46.0	26.0
2881	E 57TH AVE @ KNIGHT ST	25.0	31.422484	20.0	17.0	3.0
3569	GRANVILLE ST @ W 16TH AVE	24.0	31.515758	25.0	16.0	9.0
3589	GRANVILLE ST @ W 49TH AVE	24.0	24.566036	25.0	37.0	12.0
2406	E 33RD AVE @ KNIGHT ST	24.0	21.694272	25.0	45.0	20.0
2710	E 49TH AVE @ MAIN ST	23.0	19.470712	28.0	72.0	44.0
3596	GRANVILLE ST @ W 57TH AVE	23.0	16.120659	28.0	86.0	58.0
2070	E 1ST AVE @ NANAIMO ST	23.0	22.888913	28.0	43.0	15.0
1880	E 12TH AVE @ MAIN ST	23.0	21.274915	28.0	50.0	22.0
1241	CLARK DR @ E 12TH AVE	23.0	23.046931	28.0	42.0	14.0
1089	CASSIAR CONNECTR @ E HASTINGS ST	23.0	21.894984	28.0	44.0	16.0
2724	E 49TH AVE @ TYNE ST	23.0	17.862437	28.0	84.0	56.0

Since the client ultimately reviews 30 intersections, let's examine whether the model at least gets the true high-risk locations near the top.

```
In [80]: actual_top30 = set(rf_results.nlargest(30, "Target")["Intersection"])

for k in [30, 40, 50, 75, 100]:
    predicted_top_k = set(rf_results.nlargest(k, "Predicted")["Intersection"])

    recall = len(actual_top30 & predicted_top_k) / 30

    print(f"Recall@{k}: {recall:.1%}")
```

Recall@30: 56.7%
Recall@40: 70.0%
Recall@50: 86.7%
Recall@75: 100.0%
Recall@100: 100.0%

Looking at the recalls above, the RF is much better at identifying the general high-risk pool rather than the 17/30 initially suggested.

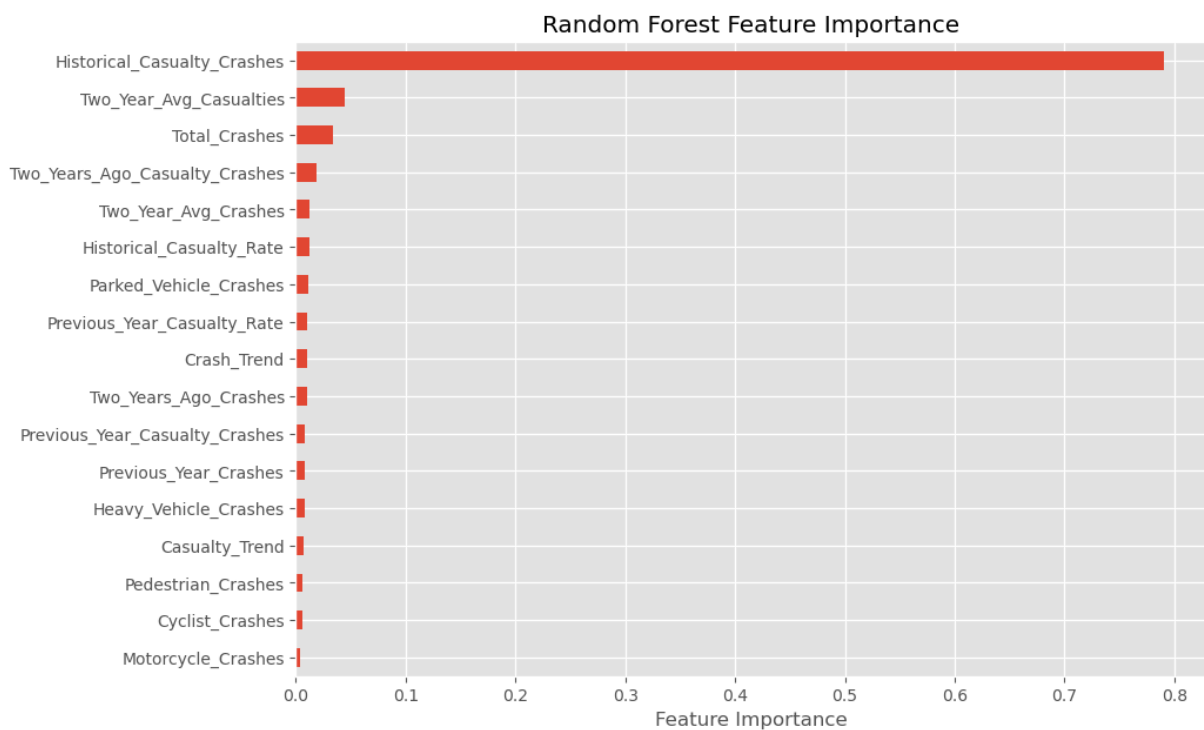
Most of the actual high-risk intersections aren't being ranked hundreds or thousands of places away. The model struggles with precise ordering near the top.

BOUNDARY RD @ MARINE WAY is actual rank 3 but predicted 53, E BROADWAY @ RUPERT ST is actual rank 14 but predicted 74, and GRANVILLE ST @ W 57TH AVE is actual rank 28 but predicted 86. Those are genuine misses worth understanding. On the other hand, JOYCE ST @ KINGSWAY is actual #2 and predicted #31

Examining feature importance

```
In [81]: feature_importance = pd.DataFrame(  
        {"Feature": feature_cols, "Importance": rf.feature_importances_  
        }).sort_values("Importance", ascending=False)  
  
        display(feature_importance)  
  
        feature_importance.sort_values("Importance").plot.barh(  
            x="Feature", y="Importance", figsize=(10, 7), legend=False  
        )  
  
        plt.xlabel("Feature Importance")  
        plt.ylabel("")  
        plt.title("Random Forest Feature Importance")  
        plt.show()
```

	Feature	Importance
1	Historical_Casualty_Crashes	0.789978
14	Two_Year_Avg_Casualties	0.043954
0	Total_Crashes	0.033462
11	Two_Years_Ago_Casualty_Crashes	0.018362
13	Two_Year_Avg_Crashes	0.012056
2	Historical_Casualty_Rate	0.011813
7	Parked_Vehicle_Crashes	0.011733
12	Previous_Year_Casualty_Rate	0.010504
15	Crash_Trend	0.010299
10	Two_Years_Ago_Crashes	0.009830
9	Previous_Year_Casualty_Crashes	0.008507
8	Previous_Year_Crashes	0.008364
6	Heavy_Vehicle_Crashes	0.008119
16	Casualty_Trend	0.006858
3	Pedestrian_Crashes	0.006129
4	Cyclist_Crashes	0.005980
5	Motorcycle_Crashes	0.004050



Historical_Casualty_Crashes accounts for ~79% of the Random Forest's feature importance. Everything else is dramatically smaller.

The RF has learned that: Intersections that had lots of casualty crashes historically will probably have lots of casualty crashes next year.

This leads me to think that there is a feature design issue. Historical_Casualty_Crashes is a cumulative count (2018 + 2019 + 2020 + 2021)m but when predicting 2020 during training, it's only 2018 + 2019.

Several of the features are also redundant:

- Historical_Casualty_Crashes
- Previous_Year_Casualty_Crashes
- Two_Years_Ago_Casualty_Crashes
- Two_Year_Avg_Casualties
- Casualty_Trend

Next steps would be to compare compare cumulative-history features against a fixed recent-history window.

2.6.2 Random Forest Regressor - Fixed two-year window

```
In [82]: fixed_window_features = [  
    "Previous_Year_Crashes",  
    "Previous_Year_Casualty_Crashes",  
    "Two_Years_Ago_Crashes",  
    "Two_Years_Ago_Casualty_Crashes",  
    "Previous_Year_Casualty_Rate",  
    "Two_Year_Avg_Crashes",  
    "Two_Year_Avg_Casualties",  
    "Crash_Trend",  
    "Casualty_Trend",  
]
```

We've removed:

- Total_Crashes
- Historical_Casualty_Crashes
- Historical_Casualty_Rate
- Pedestrian_Crashes
- Cyclist_Crashes
- Motorcycle_Crashes

- Heavy_Vehicle_Crashes
- Parked_Vehicle_Crashes

The first three are intentionally removed because they're cumulative.

```
In [83]: X_train_fixed = training_data[fixed_window_features]
y_train_fixed = training_data["Target"]

X_test_fixed = test_data[fixed_window_features]
y_test_fixed = test_data["Target"]
```

```
In [84]: rf_fixed = RandomForestRegressor(
    n_estimators=300, random_state=42, n_jobs=-1, min_samples_leaf=2
)

rf_fixed.fit(X_train_fixed, y_train_fixed)

fixed_predictions = rf_fixed.predict(X_test_fixed)
```

Evaluating

```
In [85]: fixed_mae = mean_absolute_error(y_test_fixed, fixed_predictions)

fixed_rmse = np.sqrt(mean_squared_error(y_test_fixed, fixed_predictions))

print(f"Fixed-Window RF MAE: {fixed_mae:.3f}")
print(f"Fixed-Window RF RMSE: {fixed_rmse:.3f}")
```

Fixed-Window RF MAE: 0.842
Fixed-Window RF RMSE: 1.980

```
In [86]: fixed_results = test_data[["Intersection", "Target"]].copy()

fixed_results["Predicted"] = fixed_predictions

actual_top30 = set(fixed_results.nlargest(30, "Target")["Intersection"])
predicted_top30 = set(fixed_results.nlargest(30, "Predicted")["Intersection"])

fixed_overlap = len(actual_top30 & predicted_top30)

print(
    f"Fixed-Window RF Top-30 overlap: "
    f"{fixed_overlap}/30 "
    f"({fixed_overlap / 30:.1%})"
)
```

Fixed-Window RF Top-30 overlap: 16/30 (53.3%)

```
In [87]: for k in [30, 40, 50, 75, 100]:
    predicted_top_k = set(fixed_results.nlargest(k, "Predicted")["Intersection"])

    recall = len(actual_top30 & predicted_top_k) / 30
```

```
print(f"Recall@{k}: {recall:.1%}")
```

```
Recall@30: 53.3%
Recall@40: 66.7%
Recall@50: 76.7%
Recall@75: 90.0%
Recall@100: 100.0%
```

Fixed-Window Feature Experiment results

A second Random Forest model was evaluated using only features derived from the previous two years, removing cumulative historical crash measures. This was intended to test whether recent crash patterns provided stronger predictive information than an intersection's full historical record.

The fixed-window model performed worse than the cumulative-history model across MAE, RMSE, Top-30 overlap, and Recall@K. This suggests that longer-term historical crash patterns provide useful predictive information for identifying high-risk intersections. Therefore, the cumulative-history feature set was retained for subsequent modeling.

2.6.3 Tune Random Forest

```
In [88]: train_2020 = forecast_2020
        val_2021 = forecast_2021

        X_tune_train = train_2020[feature_cols]
        y_tune_train = train_2020["Target"]

        X_tune_val = val_2021[feature_cols]
        y_tune_val = val_2021["Target"]
```

```
In [89]: params = [
        {
            "n_estimators": 300,
            "max_depth": None,
            "min_samples_leaf": 1,
            "max_features": 1.0,
        },
        {
            "n_estimators": 300,
            "max_depth": None,
            "min_samples_leaf": 2,
            "max_features": 1.0,
        },
        {
            "n_estimators": 500,
            "max_depth": None,
            "min_samples_leaf": 2,
            "max_features": 0.7,
        },
        {"n_estimators": 500, "max_depth": 15, "min_samples_leaf": 2, "max_features": 0
```

```

{"n_estimators": 500, "max_depth": 20, "min_samples_leaf": 3, "max_features": 0
]

```

```

In [90]: tuning_results = []

for p in params:
    model = RandomForestRegressor(**p, random_state=42, n_jobs=-1)

    model.fit(X_tune_train, y_tune_train)

    pred = model.predict(X_tune_val)

    mae = mean_absolute_error(y_tune_val, pred)
    rmse = np.sqrt(mean_squared_error(y_tune_val, pred))

    temp = val_2021[["Intersection", "Target"]].copy()
    temp["Predicted"] = pred

    actual_top30 = set(temp.nlargest(30, "Target")["Intersection"])

    predicted_top30 = set(temp.nlargest(30, "Predicted")["Intersection"])

    top30 = len(actual_top30 & predicted_top30)

    tuning_results.append({**p, "MAE": mae, "RMSE": rmse, "Top30": top30})

tuning_results = pd.DataFrame(tuning_results)

display(tuning_results.sort_values(["Top30", "RMSE"], ascending=[False, True]))

```

	n_estimators	max_depth	min_samples_leaf	max_features	MAE	RMSE	Top30
4	500	20.0	3	0.7	0.754184	1.577905	17
2	500	NaN	2	0.7	0.757950	1.578923	17
3	500	15.0	2	0.7	0.756681	1.578030	16
1	300	NaN	2	1.0	0.764292	1.600984	16
0	300	NaN	1	1.0	0.775869	1.624240	15

Using the best hyperparameters, lets retrain using all available pre-2022 training samples:

```

In [91]: tuned_rf = RandomForestRegressor(
    n_estimators=500,
    max_depth=20,
    min_samples_leaf=3,
    max_features=0.7,
    random_state=42,
    n_jobs=-1,
)

tuned_rf.fit(training_data[feature_cols], training_data["Target"])

```

```
tuned_predictions = tuned_rf.predict(test_data[feature_cols])
```

Evaluating vs 2022

```
In [92]: tuned_mae = mean_absolute_error(test_data["Target"], tuned_predictions)

tuned_rmse = np.sqrt(mean_squared_error(test_data["Target"], tuned_predictions))

tuned_results = test_data[["Intersection", "Target"]].copy()

tuned_results["Predicted"] = tuned_predictions

actual_top30 = set(tuned_results.nlargest(30, "Target")["Intersection"])

predicted_top30 = set(tuned_results.nlargest(30, "Predicted")["Intersection"])

tuned_overlap = len(actual_top30 & predicted_top30)

print(f"Tuned RF MAE: {tuned_mae:.3f}")
print(f"Tuned RF RMSE: {tuned_rmse:.3f}")
print(f"Tuned RF Top-30 overlap: " f"{tuned_overlap}/30 ({tuned_overlap / 30:.1%})"
```

Tuned RF MAE: 0.775

Tuned RF RMSE: 1.546

Tuned RF Top-30 overlap: 16/30 (53.3%)

```
In [93]: for k in [30, 40, 50, 75, 100]:
    predicted_top_k = set(tuned_results.nlargest(k, "Predicted")["Intersection"])

    recall = len(actual_top30 & predicted_top_k) / 30

    print(f"Recall@{k}: {recall:.1%}")
```

Recall@30: 53.3%

Recall@40: 70.0%

Recall@50: 86.7%

Recall@75: 100.0%

Recall@100: 100.0%

Summary

Random Forest hyperparameters were selected using the earlier forecasting periods, leaving 2022 as the held-out test year. Top-30 overlap decreased slightly from 17 to 16 intersections, while Recall@50 remained 86.7% and Recall@75 remained 100%.

These results indicate that tuning improved overall casualty-count prediction while maintaining similar high-risk ranking performance. The tuned Random Forest was therefore retained as the final Random Forest specification.

2.7 Second model: Gradient Boosting Regressor


```
In [94]: from sklearn.ensemble import HistGradientBoostingRegressor
```

```
In [95]: gb = HistGradientBoostingRegressor(
    learning_rate=0.05,
    max_iter=300,
    max_leaf_nodes=31,
    min_samples_leaf=20,
    l2_regularization=1.0,
    random_state=42,
)

gb.fit(X_train, y_train)

gb_pred = gb.predict(X_test)

# casualty crash counts cannot be negative
gb_pred = np.maximum(gb_pred, 0)

gb_mae = mean_absolute_error(y_test, gb_pred)
gb_rmse = np.sqrt(mean_squared_error(y_test, gb_pred))

print(f"Gradient Boosting MAE: {gb_mae:.3f}")
print(f"Gradient Boosting RMSE: {gb_rmse:.3f}")
```

Gradient Boosting MAE: 0.802

Gradient Boosting RMSE: 1.702

```
In [96]: gb_results = test_data[["Intersection", "Target"]].copy()

gb_results["Predicted"] = gb_pred

actual_top30 = set(gb_results.nlargest(30, "Target")["Intersection"])

gb_predicted_top30 = set(gb_results.nlargest(30, "Predicted")["Intersection"])

gb_top30_overlap = len(actual_top30 & gb_predicted_top30)

print(
    f"Gradient Boosting Top-30 overlap: "
    f"{gb_top30_overlap}/30 "
    f"({gb_top30_overlap / 30:.1%})"
)
```

Gradient Boosting Top-30 overlap: 17/30 (56.7%)

```
In [97]: comparison = pd.DataFrame(
    {
        "Model": ["Historical Baseline", "Random Forest", "Gradient Boosting"],
        "MAE": [baseline_mae, tuned_mae, gb_mae],
        "RMSE": [baseline_rmse, tuned_rmse, gb_rmse],
        "Top_30_Overlap": [baseline_overlap, tuned_overlap, gb_top30_overlap],
    }
)

comparison
```

Out[97]:

	Model	MAE	RMSE	Top_30_Overlap
0	Historical Baseline	0.811692	1.747720	16
1	Random Forest	0.774630	1.546464	16
2	Gradient Boosting	0.802126	1.701877	17

So far, Baseline wins MAE, Random Forest RMSE, and RF and Gradient Boosting on top 30.

Both models are only barely beating the baseline at the client's task.

2.5 Model Selection

Random Forest and Gradient Boosting regression were compared against a historical baseline for predicting casualty crashes at Vancouver intersections. Gradient Boosting achieved the highest strict Top-30 overlap, identifying 17 of the actual Top-30 intersections compared with 16 for Random Forest. However, the tuned Random Forest achieved the lowest overall prediction error, with an MAE of 0.775 and RMSE of 1.546 compared with 0.802 and 1.702 for Gradient Boosting.

The tuned Random Forest also retained strong ranking performance beyond the strict Top-30 cutoff, capturing 86.7% of the actual Top-30 intersections within its predicted Top 50 and 100% within its Top 75. Based on its stronger casualty-count accuracy while maintaining comparable high-risk ranking performance, the tuned Random Forest was selected as the final model.

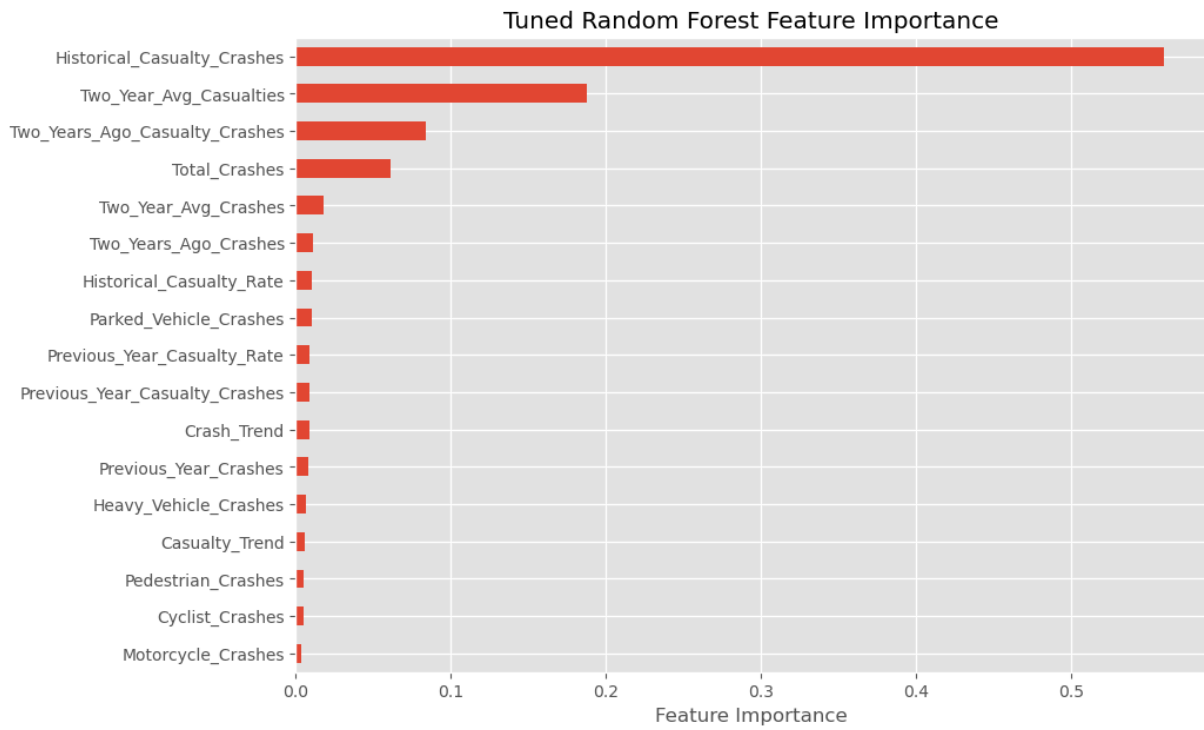
2.6 Final tuned RF feature importance

```
In [98]: # final feature importance from the tuned random forest model
final_importance = pd.DataFrame(
    {"Feature": feature_cols, "Importance": tuned_rf.feature_importances_}
).sort_values("Importance", ascending=False)

# display(final_importance)

final_importance.sort_values("Importance").plot.barh(
    x="Feature", y="Importance", figsize=(10, 7), legend=False
)

plt.xlabel("Feature Importance")
plt.ylabel("")
plt.title("Tuned Random Forest Feature Importance")
plt.show()
```



2.7 Generating 2023 features

```
In [99]: forecast_2023 = build_forecast_dataset(model_df, 2023)

print(forecast_2023.shape)

forecast_2023.head()
```

(5617, 20)

Out[99]:

	Intersection	Total_Crashes	Historical_Casualty_Crashes	Pedestrian_Crashes	Cyclist_Crashes
--	--------------	---------------	-----------------------------	--------------------	-----------------

0	29TH AVE STATION LOOP @ E 29TH AVE	1	0	0
---	---	---	---	---

1	54TH AVE AND VICTORIA LOOP @ VICTORIA DR	6	2	0
---	---	---	---	---

2	63RD AVE AND GRANVILLE LOOP @ GRANVILLE ST	6	1	0
---	---	---	---	---

3	ABBOTT ST @ EXPO BLVD	92	28	5
---	-----------------------------	----	----	---

4	ABBOTT ST @ KEEFER PL	75	9	3
---	-----------------------------	----	---	---



In [100... X_2023 = forecast_2023[feature_cols]

```
print(X_2023.shape)
print(X_2023.isna().sum())
```

```
(5617, 17)
Total_Crashes                0
Historical_Casualty_Crashes  0
Historical_Casualty_Rate     0
Pedestrian_Crashes           0
Cyclist_Crashes              0
Motorcycle_Crashes           0
Heavy_Vehicle_Crashes        0
Parked_Vehicle_Crashes       0
Previous_Year_Crashes        0
Previous_Year_Casualty_Crashes 0
Two_Years_Ago_Crashes        0
Two_Years_Ago_Casualty_Crashes 0
Previous_Year_Casualty_Rate   0
Two_Year_Avg_Crashes         0
Two_Year_Avg_Casualties      0
Crash_Trend                  0
Casualty_Trend               0
dtype: int64
```

2.8 Retraining the RF using everything until 2022

```
In [101... final_training_data = pd.concat(
    [
        forecast_2020,
        forecast_2021,
        forecast_2022,
    ],
    ignore_index=True,
)

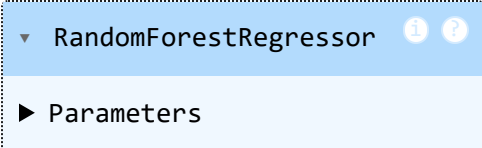
X_final = final_training_data[feature_cols]
y_final = final_training_data["Target"]

print("Final training samples:", len(final_training_data))
```

Final training samples: 14968

```
In [102... # train a final random forest model on all available data (2020-2022) to predict 20
final_rf = RandomForestRegressor(
    n_estimators=500,
    max_depth=20,
    min_samples_leaf=3,
    max_features=0.7,
    random_state=42,
    n_jobs=-1,
)

final_rf.fit(X_final, y_final)
```

Out[102...  **RandomForestRegressor** ⓘ ?
▶ Parameters

2.9 Predicting 2023

```
In [182... # prediction from every tree in the final random forest
X_2023_array = X_2023.to_numpy()

tree_predictions = np.array(
    [tree.predict(X_2023_array) for tree in final_rf.estimators_]
)

tree_predictions.shape
```

Out[182... (500, 5617)

```
In [104... forecast_2023["Predicted_Casualty_Crashes_2023"] = tree_predictions.mean(axis=0)

forecast_2023["Prediction_Lower"] = np.percentile(tree_predictions, 2.5, axis=0)

forecast_2023["Prediction_Upper"] = np.percentile(tree_predictions, 97.5, axis=0)
```

```
# crash counts cannot be negative
forecast_2023["Prediction_Lower"] = forecast_2023["Prediction_Lower"].clip(lower=0)
```

In [105... *# how often each intersection appears in a tree's top 30*

```
n_intersections = len(forecast_2023)

top30_counts = np.zeros(n_intersections)

for tree_pred in tree_predictions:
    top30_indices = np.argsort(tree_pred)[-30:]
    top30_counts[top30_indices] += 1

forecast_2023["Top30_Stability"] = top30_counts / len(tree_predictions)
```

In [106...

```
top30_2023 = (
    forecast_2023[
        [
            "Intersection",
            "Predicted_Casualty_Crashes_2023",
            "Top30_Stability",
            "Prediction_Lower",
            "Prediction_Upper",
        ]
    ]
    .sort_values("Predicted_Casualty_Crashes_2023", ascending=False)
    .head(30)
    .reset_index(drop=True)
)

top30_2023.index += 1
top30_2023.index.name = "Rank"

top30_2023["Top30_Stability"] = top30_2023["Top30_Stability"] * 100

display(top30_2023.round(1))
```

Rank	Intersection	Predicted_Casualty_Crashes_2023	Top30_Stability	Prediction_Lower	Prediction_Upper
1	BOUNDARY RD @ GRANDVIEW HWY	46.0	97.8	25.3	68.7
2	E 41ST AVE @ VICTORIA DR	38.1	95.0	22.5	53.7
3	CLARK DR @ E 1ST AVE	38.1	90.2	17.4	53.0
4	MAIN ST @ TERMINAL AVE	37.6	92.6	17.0	58.2
5	GRANDVIEW HWY @ RUPERT ST	37.6	92.4	21.9	52.9
6	KINGSWAY @ KNIGHT ST	37.2	91.0	19.2	53.6
7	KINGSWAY @ VICTORIA DR	37.0	91.4	15.2	57.2
8	E 41ST AVE @ KNIGHT ST	36.5	88.0	19.9	53.1
9	E 49TH AVE @ KNIGHT ST	30.9	70.2	13.7	47.0
10	JOYCE ST @ KINGSWAY	28.9	67.4	14.2	42.6
11	GRANVILLE ST @ W KING EDWARD AVE	28.7	65.0	13.6	41.4
12	KNIGHT ST @ SE MARINE DR	28.7	59.0	15.8	41.2
13	E 57TH AVE @ KNIGHT ST	27.8	63.8	13.3	37.3
14	E 49TH AVE @ VICTORIA	27.2	52.6	14.1	38.5

	Intersection	Predicted_Casualty_Crashes_2023	Top30_Stability	Prediction_Lower	Prediction_Upper
Rank					
	DR				
15	GRANVILLE ST @ W 16TH AVE	27.1	55.4	13.7	
16	CAMBIE ST @ W 12TH AVE	27.1	56.2	13.5	
17	KINGSWAY @ NANAIMO ST	27.1	55.2	14.1	
18	CAMBIE ST @ W 41ST AVE	25.8	47.8	12.2	
19	E 1ST AVE @ RENFREW ST	25.6	48.6	12.8	
20	KERR ST @ SE MARINE DR	25.6	45.6	14.3	
21	CLARK DR @ E BROADWAY	25.4	45.6	14.3	
22	E 49TH AVE @ FRASER ST	25.4	47.8	13.0	
23	E 41ST AVE @ RUPERT ST	24.9	44.4	14.2	
24	GRANVILLE ST @ W 49TH AVE	24.9	41.6	15.9	
25	DENMAN ST @ W GEORGIA ST	24.7	42.6	12.3	
26	E BROADWAY @ NANAIMO ST	24.5	37.2	16.3	
27	E 1ST AVE @ HWY 1	24.4	42.2	16.2	

	Intersection	Predicted_Casualty_Crashes_2023	Top30_Stability	Prediction_Lower	Prediction_Upper
Rank					
28	BOUNDARY RD @ MARINE WAY	24.3	42.2	14.7	33.9
29	E HASTINGS ST @ NANAIMO ST	24.3	38.2	12.8	39.6
30	E 33RD AVE @ KNIGHT ST	24.0	43.0	14.2	33.8

2023 Intersection Risk Forecast

The final Random Forest model was retrained using all available historical forecasting data through 2022 and used to estimate casualty crashes at Vancouver intersections for 2023. Intersections were ranked by their predicted casualty crash count, consistent with the client's objective of identifying approximately 30 locations for engineering review.

To assess the stability of the ranking, each of the 500 trees in the Random Forest was also used to independently rank intersections. Top-30 stability represents the percentage of trees in which an intersection appeared among the 30 highest-risk locations.

The highest-ranked locations show strong agreement across the forest. For example, Boundary Rd @ Grandview Hwy was ranked first with approximately 46 predicted casualty crashes and appeared in the Top 30 in 97.8% of trees. The first eight intersections each achieved at minimum 88% Top-30 stability.

Stability decreases further down the ranking, indicating substantially greater uncertainty around the exact composition of the lower portion of the Top 30. Thus, the ranking could more appropriately be interpreted as identifying a high-risk priority group rather than establishing a precise ordering between similarly ranked intersections.

3. Model Interpretation and Engineering Priorities

3.1 Global feature importance

```
In [107]: final_feature_importance = pd.DataFrame(
    {"Feature": feature_cols, "Importance": final_rf.feature_importances_}
).sort_values("Importance", ascending=False)

display(final_feature_importance)
```

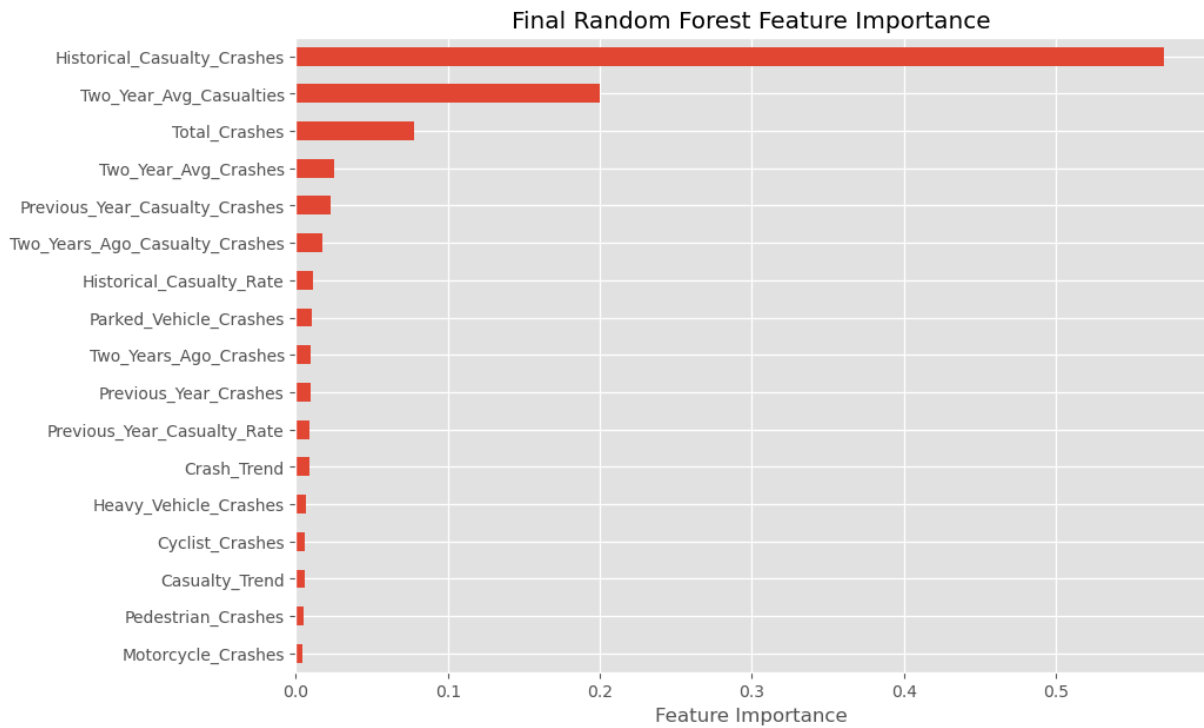
```

final_feature_importance.sort_values("Importance").plot.barh(
    x="Feature", y="Importance", figsize=(10, 7), legend=False
)

plt.xlabel("Feature Importance")
plt.ylabel("")
plt.title("Final Random Forest Feature Importance")
plt.show()

```

	Feature	Importance
1	Historical_Casualty_Crashes	0.570776
14	Two_Year_Avg_Casualties	0.199904
0	Total_Crashes	0.077746
13	Two_Year_Avg_Crashes	0.025374
9	Previous_Year_Casualty_Crashes	0.023205
11	Two_Years_Ago_Casualty_Crashes	0.017490
2	Historical_Casualty_Rate	0.011245
7	Parked_Vehicle_Crashes	0.010667
10	Two_Years_Ago_Crashes	0.009903
8	Previous_Year_Crashes	0.009399
12	Previous_Year_Casualty_Rate	0.008793
15	Crash_Trend	0.008627
6	Heavy_Vehicle_Crashes	0.006581
4	Cyclist_Crashes	0.005952
16	Casualty_Trend	0.005462
3	Pedestrian_Crashes	0.004797
5	Motorcycle_Crashes	0.004079



Feature importance was examined to understand which historical crash characteristics contributed most strongly to the Random Forest's predictions. These values describe how useful each feature was to the model across all intersections and should not be interpreted as causal effects.

Historical casualty crash frequency was the strongest predictor of future casualty crashes, indicating that intersections with persistent histories of injury-producing collisions tend to remain high-risk. Total crash volume and recent casualty history also contributed to the predictions, while characteristics such as pedestrian, cyclist, motorcycle, and heavy-vehicle involvement provided additional information about the historical crash profile of each intersection.

Global feature importance explains the model's overall behavior but does not explain why a specific intersection received a high prediction. Intersection-level characteristics are therefore examined

3.2 Intersection Characteristics (Top-30)

```
In [108... top30_characteristics = (
    forecast_2023.sort_values("Predicted_Casualty_Crashes_2023", ascending=False)
    .head(30)[
        [
            "Intersection",
            "Predicted_Casualty_Crashes_2023",
            "Historical_Casualty_Crashes",
            "Total_Crashes",
            "Historical_Casualty_Rate",
            "Previous_Year_Crashes",
```

```
        "Previous_Year_Casualty_Crashes",
        "Crash_Trend",
        "Casualty_Trend",
        "Pedestrian_Crashes",
        "Cyclist_Crashes",
        "Motorcycle_Crashes",
        "Heavy_Vehicle_Crashes",
    ]
    ].reset_index(drop=True)
)

top30_characteristics.index += 1
top30_characteristics.index.name = "Rank"

display(top30_characteristics.round(2))
```

	Intersection	Predicted_Casualty_Crashes_2023	Historical_Casualty_Crashes	Total_Crashes
Rank				
1	BOUNDARY RD @ GRANDVIEW HWY	45.99	348	1094
2	E 41ST AVE @ VICTORIA DR	38.13	176	795
3	CLARK DR @ E 1ST AVE	38.05	201	470
4	MAIN ST @ TERMINAL AVE	37.63	193	637
5	GRANDVIEW HWY @ RUPERT ST	37.58	190	735
6	KINGSWAY @ KNIGHT ST	37.20	173	518
7	KINGSWAY @ VICTORIA DR	37.00	182	598
8	E 41ST AVE @ KNIGHT ST	36.52	199	529
9	E 49TH AVE @ KNIGHT ST	30.90	161	435
10	JOYCE ST @ KINGSWAY	28.88	144	530
11	GRANVILLE ST @ W KING EDWARD AVE	28.71	143	439
12	KNIGHT ST @ SE MARINE DR	28.66	155	400
13	E 57TH AVE @ KNIGHT ST	27.78	143	474
14	E 49TH AVE @ VICTORIA	27.18	158	417

	Intersection	Predicted_Casualty_Crashes_2023	Historical_Casualty_Crashes	Total_Crashes
Rank				
	DR			
15	GRANVILLE ST @ W 16TH AVE	27.15	134	44%
16	CAMBIE ST @ W 12TH AVE	27.10	137	47%
17	KINGSWAY @ NANAIMO ST	27.05	137	40%
18	CAMBIE ST @ W 41ST AVE	25.83	138	69%
19	E 1ST AVE @ RENFREW ST	25.58	128	49%
20	KERR ST @ SE MARINE DR	25.57	139	30%
21	CLARK DR @ E BROADWAY	25.40	146	36%
22	E 49TH AVE @ FRASER ST	25.40	136	56%
23	E 41ST AVE @ RUPERT ST	24.88	141	30%
24	GRANVILLE ST @ W 49TH AVE	24.87	124	35%
25	DENMAN ST @ W GEORGIA ST	24.72	127	41%
26	E BROADWAY @ NANAIMO ST	24.49	122	32%
27	E 1ST AVE @ HWY 1	24.44	111	25%

	Intersection	Predicted_Casualty_Crashes_2023	Historical_Casualty_Crashes	Total_Crashes
Rank				
28	BOUNDARY RD @ MARINE WAY	24.33	110	316
29	E HASTINGS ST @ NANAIMO ST	24.30	146	443
30	E 33RD AVE @ KNIGHT ST	24.01	114	276

The highest-ranked locations generally have substantial historical crash volumes and casualty crash histories. However, the intersections differ in their recent trends and crash composition.

For example, Boundary Rd @ Grandview Hwy has the highest predicted casualty crash count and also has the largest historical crash volume among the highest-ranked locations. Other locations exhibit different patterns. Kingsway @ Knight St experienced increasing total and casualty crashes in the most recent year, while E 49th Ave @ Knight St experienced a substantial decline in casualty crashes despite remaining highly ranked based on its broader historical record.

These characteristics provide context for the rankings but should not be interpreted as causal explanations. The model's intersection-specific predictions are examined in the following section to identify which features contributed most strongly to individual predictions.

3.3 Top 30 Crash profiles

The forecast tells us which intersections are expected to have higher casualty crash risk, but not what might be worth investigating at each location. To add that context, I compared the crash profiles of the predicted Top 30 against Vancouver intersections overall.

```
In [127... # create crash-level dataset for engineering profiles
profile_df = model_df.copy()

# crash characteristics
profile_df["Pedestrian"] = (profile_df["Pedestrian Involved"] == "Yes").astype(int)
profile_df["Cyclist"] = (profile_df["Cyclist Involved"] == "Yes").astype(int)
profile_df["Motorcycle"] = (profile_df["Motorcycle Involved"] == "Yes").astype(int)
```

```
profile_df["Heavy_Vehicle"] = (profile_df["Heavy Veh Involved"] == "Yes").astype(int)
profile_df["Casualty"] = (profile_df["Crash Severity"] == "CASUALTY CRASH").astype(int)
```

```
In [147... # crash configuration indicators
profile_df["Rear_End"] = (
    profile_df["Derived Crash Congifuration"] == "REAR END"
).astype(int)

profile_df["Side_Impact"] = (
    profile_df["Derived Crash Congifuration"] == "SIDE IMPACT"
).astype(int)

profile_df["Single_Vehicle"] = (
    profile_df["Derived Crash Congifuration"] == "SINGLE VEHICLE"
).astype(int)

profile_df["Head_On"] = (profile_df["Derived Crash Congifuration"] == "HEAD ON").astype(int)

profile_df["Same_Direction_Sideswipe"] = (
    profile_df["Derived Crash Congifuration"] == "SIDE SWIPE - SAME DIRECTION"
).astype(int)
```

```
In [148... # Temporal features

print(profile_df["Time Category"].value_counts())
```

```
Time Category
15:00-17:59    35963
12:00-14:59    31568
09:00-11:59    23266
18:00-20:59    17635
06:00-08:59    13389
21:00-23:59     6463
00:00-02:59     2183
03:00-05:59     1307
Name: count, dtype: int64
```

```
In [149... # temporal crash characteristics

profile_df["Morning_Peak"] = (profile_df["Time Category"] == "06:00-08:59").astype(int)

profile_df["Afternoon_Peak"] = (profile_df["Time Category"] == "15:00-17:59").astype(int)

profile_df["Night"] = (
    profile_df["Time Category"].isin(
        [
            "21:00-23:59",
            "00:00-02:59",
            "03:00-05:59",
        ]
    )
)
```



```

    )
).astype(int)

profile_df["Weekend"] = (
    profile_df["Day Of Week"].isin(
        [
            "SATURDAY",
            "SUNDAY",
        ]
    )
).astype(int)

print(profile_df["Day Of Week"].value_counts())

```

```

Day Of Week
FRIDAY      22668
THURSDAY    21381
WEDNESDAY   20430
TUESDAY     20421
MONDAY      17297
SATURDAY    16945
SUNDAY      12632
Name: count, dtype: int64

```

```

In [151... profile_features = [
    # road users / vehicle types
    "Pedestrian",
    "Cyclist",
    "Motorcycle",
    "Heavy_Vehicle",
    # temporal characteristics
    "Morning_Peak",
    "Afternoon_Peak",
    "Night",
    "Weekend",
    # crash configurations
    "Rear_End",
    "Side_Impact",
    "Single_Vehicle",
    "Head_On",
    "Same_Direction_Sideswipe",
]

```

Vancouver-wide baseline to compare with

```

In [152... vancouver_rates = profile_df[profile_features].mean()

vancouver_rates_display = vancouver_rates.mul(100).round(2).rename("Vancouver_Percentage")

display(vancouver_rates_display)

```

Pedestrian	2.47
Cyclist	2.85
Motorcycle	1.09
Heavy_Vehicle	7.30
Morning_Peak	10.16
Afternoon_Peak	27.29
Night	7.55
Weekend	22.45
Rear_End	28.36
Side_Impact	20.98
Single_Vehicle	9.24
Head_On	1.07
Same_Direction_Sideswipe	11.25

Name: Vancouver_Percent, dtype: float64

Temporally, Afternoon_Peak, Weekend, and Morning_Peak are highest proportionally.

Otherwise, Rear_end and Side_Impact are highest in crash configuration.

```
In [155... intersection_rates = profile_df.groupby("Intersection")[profile_features].mean()

intersection_indices = intersection_rates / vancouver_rates

intersection_indices.columns = [f"{col}_Index" for col in intersection_indices.columns]

intersection_indices = intersection_indices.reset_index()
```

```
In [157... top30_profiles = (
    top30_2023[["Intersection"]]
    .reset_index()
    .merge(intersection_indices, on="Intersection", how="left")
)

display(top30_profiles.round(2).head())
```

	Rank	Intersection	Pedestrian_Index	Cyclist_Index	Motorcycle_Index	Heavy_Vehicle_Index
0	1	BOUNDARY RD @ GRANDVIEW HWY	0.11	0.03	0.67	1.26
1	2	E 41ST AVE @ VICTORIA DR	0.51	0.13	1.05	1.12
2	3	CLARK DR @ E 1ST AVE	0.34	1.20	1.96	1.05
3	4	MAIN ST @ TERMINAL AVE	1.86	0.56	1.31	0.85
4	5	GRANDVIEW HWY @ RUPERT ST	0.71	0.29	0.38	0.37



Restricting analysis to just the top-30

```
In [162... index_cols = [col for col in top30_profiles.columns if col.endswith("_Index")]

profile_rows = []

# extracting the top 3 characteristics for each intersection based on the index value
for _, row in top30_profiles.iterrows():

    values = row[index_cols].sort_values(ascending=False)

    top_features = values.head(3)

    profile_rows.append(
        {
            "Intersection": row["Intersection"],
            "Characteristic_1": top_features.index[0].replace("_Index", ""),
            "Index_1": top_features.iloc[0],
            "Characteristic_2": top_features.index[1].replace("_Index", ""),
            "Index_2": top_features.iloc[1],
            "Characteristic_3": top_features.index[2].replace("_Index", ""),
            "Index_3": top_features.iloc[2],
        }
    )

engineering_profiles = pd.DataFrame(profile_rows)

engineering_profiles.index += 1
engineering_profiles.index.name = "Rank"

display(engineering_profiles.round(2))
```

Rank	Intersection	Characteristic_1	Index_1	Characteristic_2	Index_2	
1	BOUNDARY RD @ GRANDVIEW HWY	Rear_End	1.76	Night	1.38	
2	E 41ST AVE @ VICTORIA DR	Heavy_Vehicle	1.12	Side_Impact	1.09	
3	CLARK DR @ E 1ST AVE	Motorcycle	1.96	Rear_End	1.73	
4	MAIN ST @ TERMINAL AVE	Pedestrian	1.86	Head_On	1.34	
5	GRANDVIEW HWY @ RUPERT ST	Head_On	1.40	Weekend	1.18	
6	KINGSWAY @ KNIGHT ST	Rear_End	1.51	Night	1.41	Sam
7	KINGSWAY @ VICTORIA DR	Night	1.46	Pedestrian	1.35	
8	E 41ST AVE @ KNIGHT ST	Heavy_Vehicle	1.90	Night	1.46	
9	E 49TH AVE @ KNIGHT ST	Head_On	1.72	Rear_End	1.64	
10	JOYCE ST @ KINGSWAY	Night	1.57	Rear_End	1.12	
11	GRANVILLE ST @ W KING EDWARD AVE	Morning_Peak	1.54	Same_Direction_Sideswipe	1.51	
12	KNIGHT ST @ SE MARINE DR	Rear_End	1.83	Heavy_Vehicle	1.68	
13	E 57TH AVE @ KNIGHT ST	Heavy_Vehicle	1.62	Rear_End	1.38	
14	E 49TH AVE @ VICTORIA	Head_On	2.28	Motorcycle	1.35	

Rank	Intersection		Characteristic_1	Index_1	Characteristic_2	Index_2	
	DR						
15	GRANVILLE ST @ W 16TH AVE		Rear_End	1.39	Weekend	1.21	
16	CAMBIE ST @ W 12TH AVE		Head_On	1.79	Single_Vehicle	1.52	
17	KINGSWAY @ NANAIMO ST		Rear_End	1.49	Weekend	1.04	
18	CAMBIE ST @ W 41ST AVE		Afternoon_Peak	1.12	Side_Impact	1.04	
19	E 1ST AVE @ RENFREW ST		Weekend	1.40	Rear_End	1.18	
20	KERR ST @ SE MARINE DR		Rear_End	1.93	Morning_Peak	1.39	
21	CLARK DR @ E BROADWAY		Motorcycle	2.50	Heavy_Vehicle	1.63	
22	E 49TH AVE @ FRASER ST	Same_Direction_Sideswipe		1.15	Side_Impact	1.15	
23	E 41ST AVE @ RUPERT ST		Morning_Peak	1.84	Night	1.46	
24	GRANVILLE ST @ W 49TH AVE	Same_Direction_Sideswipe		1.24	Night	1.22	
25	DENMAN ST @ W GEORGIA ST		Head_On	2.05	Rear_End	1.59	Sam
26	E BROADWAY @ NANAIMO ST		Motorcycle	1.68	Morning_Peak	1.50	
27	E 1ST AVE @ HWY 1		Rear_End	2.13	Morning_Peak	1.33	

	Intersection	Characteristic_1	Index_1	Characteristic_2	Index_2
Rank					
28	BOUNDARY RD @ MARINE WAY	Heavy_Vehicle	2.21	Rear_End	1.81
29	E HASTINGS ST @ NANAIMO ST	Heavy_Vehicle	2.04	Pedestrian	1.64
30	E 33RD AVE @ KNIGHT ST	Head_On	2.04	Morning_Peak	1.60

To flag something as notable, lets compare against the 1.25 x Vancouver avg.

```
In [163... notable_profiles = engineering_profiles.copy()

for i in range(1, 4):
    mask = notable_profiles[f"Index_{i}"] < 1.25
    notable_profiles.loc[mask, [f"Characteristic_{i}", f"Index_{i}"]] = None
display(notable_profiles.round(2))
```

Rank	Intersection	Characteristic_1	Index_1	Characteristic_2	Index_2	Char
1	BOUNDARY RD @ GRANDVIEW HWY	Rear_End	1.76	Night	1.38	H
2	E 41ST AVE @ VICTORIA DR	None	NaN	None	NaN	
3	CLARK DR @ E 1ST AVE	Motorcycle	1.96	Rear_End	1.73	
4	MAIN ST @ TERMINAL AVE	Pedestrian	1.86	Head_On	1.34	
5	GRANDVIEW HWY @ RUPERT ST	Head_On	1.40	None	NaN	
6	KINGSWAY @ KNIGHT ST	Rear_End	1.51	Night	1.41	
7	KINGSWAY @ VICTORIA DR	Night	1.46	Pedestrian	1.35	
8	E 41ST AVE @ KNIGHT ST	Heavy_Vehicle	1.90	Night	1.46	
9	E 49TH AVE @ KNIGHT ST	Head_On	1.72	Rear_End	1.64	
10	JOYCE ST @ KINGSWAY	Night	1.57	None	NaN	
11	GRANVILLE ST @ W KING EDWARD AVE	Morning_Peak	1.54	Same_Direction_Sideswipe	1.51	H
12	KNIGHT ST @ SE MARINE DR	Rear_End	1.83	Heavy_Vehicle	1.68	M
13	E 57TH AVE @ KNIGHT ST	Heavy_Vehicle	1.62	Rear_End	1.38	
14	E 49TH AVE @ VICTORIA	Head_On	2.28	Motorcycle	1.35	

Rank	Intersection	Characteristic_1	Index_1	Characteristic_2	Index_2	Char
	DR					
15	GRANVILLE ST @ W 16TH AVE	Rear_End	1.39	None	NaN	
16	CAMBIE ST @ W 12TH AVE	Head_On	1.79	Single_Vehicle	1.52	
17	KINGSWAY @ NANAIMO ST	Rear_End	1.49	None	NaN	
18	CAMBIE ST @ W 41ST AVE	None	NaN	None	NaN	
19	E 1ST AVE @ RENFREW ST	Weekend	1.40	None	NaN	
20	KERR ST @ SE MARINE DR	Rear_End	1.93	Morning_Peak	1.39	
21	CLARK DR @ E BROADWAY	Motorcycle	2.50	Heavy_Vehicle	1.63	
22	E 49TH AVE @ FRASER ST	None	NaN	None	NaN	
23	E 41ST AVE @ RUPERT ST	Morning_Peak	1.84	Night	1.46	
24	GRANVILLE ST @ W 49TH AVE	None	NaN	None	NaN	
25	DENMAN ST @ W GEORGIA ST	Head_On	2.05	Rear_End	1.59	Same_Directic
26	E BROADWAY @ NANAIMO ST	Motorcycle	1.68	Morning_Peak	1.50	
27	E 1ST AVE @ HWY 1	Rear_End	2.13	Morning_Peak	1.33	Aft

	Intersection	Characteristic_1	Index_1	Characteristic_2	Index_2	Cha
Rank						
28	BOUNDARY RD @ MARINE WAY	Heavy_Vehicle	2.21	Rear_End	1.81	M
29	E HASTINGS ST @ NANAIMO ST	Heavy_Vehicle	2.04	Pedestrian	1.64	
30	E 33RD AVE @ KNIGHT ST	Head_On	2.04	Morning_Peak	1.60	

Final profile of the top30

```
In [164... final_engineering_profiles = (
    top30_2023[
        [
            "Intersection",
            "Predicted_Casualty_Crashes_2023",
            "Top30_Stability",
        ]
    ]
    .reset_index()
    .merge(notable_profiles.reset_index(), on="Intersection", how="left")
)

display(final_engineering_profiles.round(2))
```

Rank_x	Intersection	Predicted_Casualty_Crashes_2023	Top30_Stability	Rank_y	Character
0	1 BOUNDARY RD @ GRANDVIEW HWY	45.99	97.8	1	Re
1	2 E 41ST AVE @ VICTORIA DR	38.13	95.0	2	
2	3 CLARK DR @ E 1ST AVE	38.05	90.2	3	Mot
3	4 MAIN ST @ TERMINAL AVE	37.63	92.6	4	Pec
4	5 GRANDVIEW HWY @ RUPERT ST	37.58	92.4	5	He
5	6 KINGSWAY @ KNIGHT ST	37.20	91.0	6	Re
6	7 KINGSWAY @ VICTORIA DR	37.00	91.4	7	
7	8 E 41ST AVE @ KNIGHT ST	36.52	88.0	8	Heavy_
8	9 E 49TH AVE @ KNIGHT ST	30.90	70.2	9	He
9	10 JOYCE ST @ KINGSWAY	28.88	67.4	10	
10	11 GRANVILLE ST @ W KING EDWARD AVE	28.71	65.0	11	Mornin
11	12 KNIGHT ST @ SE MARINE DR	28.66	59.0	12	Re
12	13 E 57TH AVE @ KNIGHT ST	27.78	63.8	13	Heavy_
13	14 E 49TH AVE @ VICTORIA DR	27.18	52.6	14	He

	Rank_x	Intersection	Predicted_Casualty_Crashes_2023	Top30_Stability	Rank_y	Character
14	15	GRANVILLE ST @ W 16TH AVE	27.15	55.4	15	Re
15	16	CAMBIE ST @ W 12TH AVE	27.10	56.2	16	He
16	17	KINGSWAY @ NANAIMO ST	27.05	55.2	17	Re
17	18	CAMBIE ST @ W 41ST AVE	25.83	47.8	18	
18	19	E 1ST AVE @ RENFREW ST	25.58	48.6	19	W
19	20	KERR ST @ SE MARINE DR	25.57	45.6	20	Re
20	21	CLARK DR @ E BROADWAY	25.40	45.6	21	Mot
21	22	E 49TH AVE @ FRASER ST	25.40	47.8	22	
22	23	E 41ST AVE @ RUPERT ST	24.88	44.4	23	Mornin
23	24	GRANVILLE ST @ W 49TH AVE	24.87	41.6	24	
24	25	DENMAN ST @ W GEORGIA ST	24.72	42.6	25	He
25	26	E BROADWAY @ NANAIMO ST	24.49	37.2	26	Mot
26	27	E 1ST AVE @ HWY 1	24.44	42.2	27	Re
27	28	BOUNDARY RD @	24.33	42.2	28	Heavy_

	Rank_x	Intersection	Predicted_Casualty_Crashes_2023	Top30_Stability	Rank_y	Character
		MARINE WAY				
28	29	E HASTINGS ST @ NANAIMO ST	24.30	38.2	29	Heavy_
29	30	E 33RD AVE @ KNIGHT ST	24.01	43.0	30	He

What kinds of crashes historically characterize the intersections that the model predicts will be highest-risk in 2023?

Crash profiles were constructed using road-user and vehicle involvement, temporal characteristics, and crash configuration. These included pedestrian, cyclist, motorcycle, and heavy-vehicle involvement; morning peak, afternoon peak, nighttime, and weekend crashes; and common crash configurations such as rear-end, side-impact, single-vehicle, head-on, and same-direction sideswipe crashes.

For each characteristic, a relative prevalence index was calculated by dividing its proportion of crashes at an individual intersection by its proportion across all Vancouver intersection crashes. An index of 1.0 therefore represents the Vancouver-wide baseline, while an index of 1.5 indicates that the characteristic is 50% more prevalent at that intersection. Only characteristics with an index of at least 1.25 were highlighted, and a maximum of three characteristics were retained for each intersection.

The results above suggest that predicted high-risk intersections do not share a single crash profile.

E.g. At Boundary Rd @ Grandview Hwy, rear-end crashes are 1.76 times as prevalent as the Vancouver baseline, accompanied by elevated nighttime (1.38×) and heavy-vehicle (1.26×) involvement. Knight St @ SE Marine Dr similarly shows elevated rear-end crashes (1.83×), heavy-vehicle involvement (1.68×), and morning-peak crashes (1.43×). At E 1st Ave @ Hwy 1, rear-end crashes are more than twice as prevalent as the citywide baseline (2.13×).

Main St @ Terminal Ave has pedestrian involvement at 1.86 times the Vancouver baseline, while Clark Dr @ E Broadway has motorcycle involvement at 2.50 times the baseline and heavy-vehicle involvement at 1.63 times the baseline. Boundary Rd @ Marine Way is particularly characterized by heavy-vehicle involvement (2.21×), together with elevated rear-end and morning-peak crashes (both 1.81×). E Hastings St @ Nanaimo St also has elevated heavy-vehicle (2.04×) and pedestrian (1.64×) involvement.

Head-on crashes are disproportionately represented at locations including E 49th Ave @ Victoria Dr (2.28×), Denman St @ W Georgia St (2.05×), and E 33rd Ave @ Knight St (2.04×). Meanwhile, Granville St @ W King Edward Ave shows elevated same-direction sideswipe crashes (1.51×) alongside morning-peak (1.54×) and heavy-vehicle (1.39×) activity.

Some predicted priority intersections have no crash characteristic exceeding the 1.25 threshold (as reflected as Nan). This does not indicate that these locations are low risk; rather, their historical crash composition is relatively similar to the Vancouver-wide distribution despite their high predicted casualty crash counts.

It should be noted that these profiles are descriptive rather than causal.

For example, a higher rate of nighttime crashes does not necessarily mean that poor lighting is causing crashes, but it does suggest that lighting and visibility may be worth investigating. Similarly, higher pedestrian involvement could point engineers toward crossing conditions or turning conflicts, while heavy-vehicle involvement could suggest looking at vehicle movements or intersection geometry. Patterns in rear-end or head-on crashes could also help identify traffic flow or conflict issues. Overall, the forecast ranking helps identify which intersections should be prioritized, while the crash profiles provide additional context on what engineers should look for when reviewing each location.

Splitting the profile into tiers via a priority score

Priority Score=0.50(Forecast Risk)+0.30(Stability)+0.20(Crash Profile Strength)

- 50% Forecast risk - is the main objective of the model
- 30% Stability - as predictions that consistently remain in the Top 30 should be rewarded
- 20% Profile strength - reward locations with a particularly overrepresented crash characteristic

```
In [171... # strongest overrepresented crash characteristic
final_engineering_profiles["Profile_Strength"] = (
    final_engineering_profiles[["Index_1", "Index_2", "Index_3"]]
    .max(axis=1)
    .fillna(1.0)
)
```

```
In [172... from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

final_engineering_profiles[["Risk_Score", "Stability_Score", "Profile_Score"]] = (
    scaler.fit_transform(
        final_engineering_profiles[
            [
                "Predicted_Casualty_Crashes_2023",
                "Top30_Stability",
```

```

        "Profile_Strength",
    ]
)
)

```

```

In [177... # calculate the composite score
final_engineering_profiles["Priority_Score"] = (
    0.50 * final_engineering_profiles["Risk_Score"]
    + 0.30 * final_engineering_profiles["Stability_Score"]
    + 0.20 * final_engineering_profiles["Profile_Score"]
)

final_engineering_profiles["Priority_Score"] *= 100

```

```

In [178... priority_results = final_engineering_profiles.sort_values(
    "Priority_Score", ascending=False
).reset_index(drop=True)

priority_results["Priority_Rank"] = priority_results.index + 1

```

```

In [179... display(
    priority_results[
        [
            "Priority_Rank",
            "Intersection",
            "Priority_Score",
            "Predicted_Casualty_Crashes_2023",
            "Top30_Stability",
            "Profile_Strength",
            "Characteristic_1",
            "Index_1",
        ]
    ].round(2)
)

```

Priority_Rank	Intersection	Priority_Score	Predicted_Casualty_Crashes_2023	Top30_Stabili
0	1 BOUNDARY RD @ GRANDVIEW HWY	90.20	45.99	97
1	2 CLARK DR @ E 1ST AVE	71.02	38.05	90
2	3 MAIN ST @ TERMINAL AVE	69.87	37.63	92
3	4 E 41ST AVE @ KNIGHT ST	65.68	36.52	88
4	5 GRANDVIEW HWY @ RUPERT ST	63.54	37.58	92
5	6 KINGSWAY @ KNIGHT ST	63.46	37.20	91
6	7 KINGSWAY @ VICTORIA DR	62.53	37.00	91
7	8 E 41ST AVE @ VICTORIA DR	60.73	38.13	95
8	9 E 49TH AVE @ KNIGHT ST	41.59	30.90	70
9	10 JOYCE ST @ KINGSWAY	33.68	28.88	67
10	11 KNIGHT ST @ SE MARINE DR	32.49	28.66	59
11	12 E 49TH AVE @ VICTORIA DR	31.96	27.18	52
12	13 GRANVILLE ST @ W KING EDWARD AVE	31.64	28.71	65
13	14 E 57TH AVE @ KNIGHT ST	30.00	27.78	63

Priority_Rank	Intersection	Priority_Score	Predicted_Casualty_Crashes_2023	Top30_Stabili
14	CLARK DR @ E BROADWAY	27.32	25.40	45
15	CAMBIE ST @ W 12TH AVE	27.02	27.10	56
16	KINGSWAY @ NANAIMO ST	22.33	27.05	55
17	GRANVILLE ST @ W 16TH AVE	21.42	27.15	55
18	KERR ST @ SE MARINE DR	20.14	25.57	45
19	BOUNDARY RD @ MARINE WAY	19.36	24.33	42
20	E 1ST AVE @ HWY 1	18.51	24.44	42
21	DENMAN ST @ W GEORGIA ST	18.29	24.72	42
22	E 33RD AVE @ KNIGHT ST	16.75	24.01	43
23	E 41ST AVE @ RUPERT ST	16.73	24.88	44
24	E HASTINGS ST @ NANAIMO ST	15.05	24.30	38
25	E 1ST AVE @ RENFREW ST	14.58	25.58	48
26	E BROADWAY @ NANAIMO ST	10.18	24.49	37

	Priority_Rank	Intersection	Priority_Score	Predicted_Casualty_Crashes_2023	Top30_Stabili
27	28	CAMBIE ST @ W 41ST AVE	9.37	25.83	47
28	29	E 49TH AVE @ FRASER ST	8.40	25.40	47
29	30	GRANVILLE ST @ W 49TH AVE	4.14	24.87	41

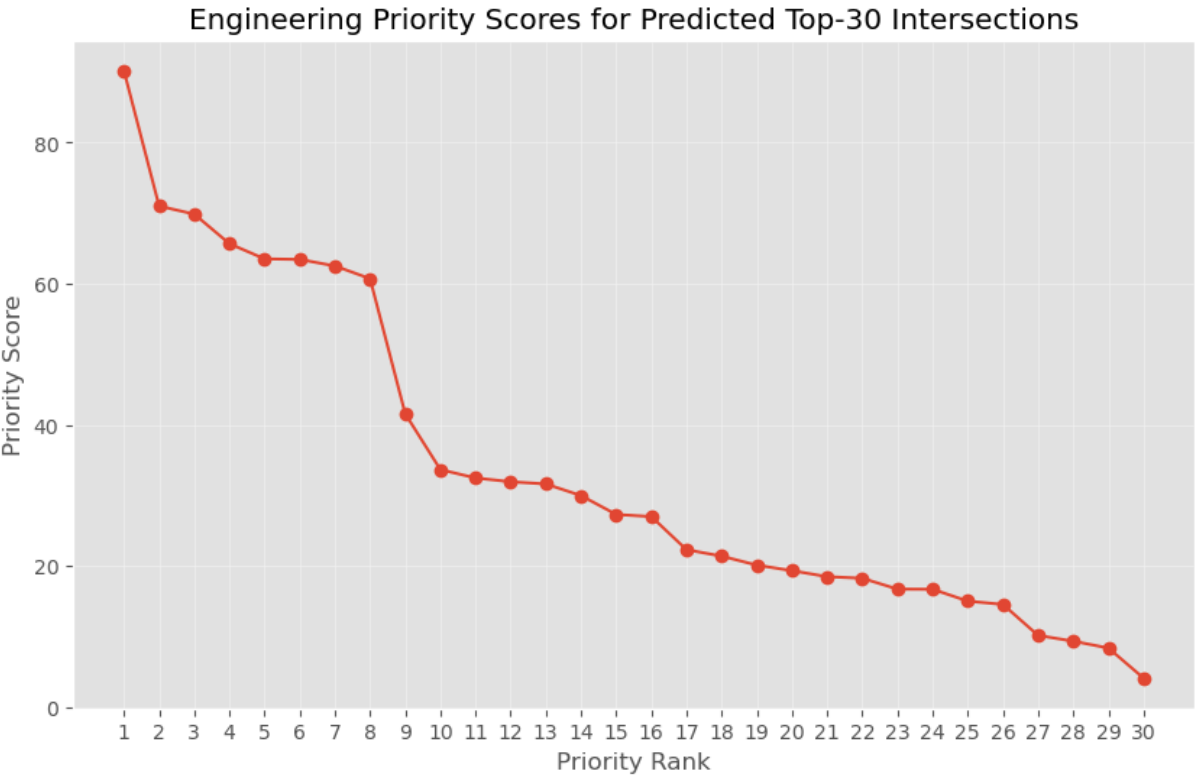
```
In [181... plt.figure(figsize=(10, 6))

plt.plot(
    priority_results["Priority_Rank"], priority_results["Priority_Score"], marker="
)

plt.xlabel("Priority Rank")
plt.ylabel("Priority Score")
plt.title("Engineering Priority Scores for Predicted Top-30 Intersections")

plt.xticks(range(1, 31))
plt.grid(alpha=0.3)

plt.show()
```



The priority scores show a clear separation between the first eight intersections and the rest of the predicted Top 30. Scores remain above 60 through rank 8, before dropping from 60.73 to 41.59 between ranks 8 and 9. Rather than selecting an arbitrary score threshold, this natural break was used to identify the highest-priority group for engineering review.

The priority score's weights are an estimation and simply acts as a suggestion, however the drop in score is indicative of an immediate 'tier' of intersections that should be reviewed.

4. Recommended Engineering Priorities

4.1 Summary of findings

This analysis used historical Vancouver crash data to identify intersections that may be higher priorities for engineering review. After cleaning and normalizing the intersection data, the Random Forest was selected then fine tuned. The tuned Random Forest performed best overall, with an MAE of **0.775** and RMSE of **1.546**, improving on the historical baseline. It identified **16/30** of the actual Top 30 intersections, increasing to **70% recall within the Top 40** and **86.7% within the Top 50**.

The final model was then used to predict casualty crashes for 2023. **Boundary Rd @ Grandview Hwy** was ranked first, with around **46 predicted casualty crashes** and **97.8% Top-30 stability**. Forecast stability measures how consistently an intersection appears in the predicted Top 30 across repeated model predictions. For each run, the Random Forest was refit/resampled and the percentage of runs where an intersection remained in the Top 30 was recorded. For example, a stability of 97.8% means the intersection appeared in the Top 30 in 97.8% of runs. Higher stability suggests that an intersection's priority ranking is less sensitive to variation in the model.

The predicted Top 30 were then profiled to find crash characteristics that occur more often at each intersection compared relatively to the Vancouver average, with a weighting of 1.25x the Vancouver wide average being flagged.

Overall, the results give us two main things: **which intersections should be looked at first, and what crash patterns may be worth investigating at each one**. The following section uses these results to group intersections into different engineering priority levels.

4.2 Flagged Intersections

To combine the results into a final engineering priority ranking, a composite priority score was calculated for each of the predicted Top 30 intersections. The score combines three parts of the analysis: the predicted number of casualty crashes, forecast stability, and the strength of the most overrepresented crash characteristic at the intersection. These

represent the overall predicted risk, confidence in the ranking, and whether there is a specific crash pattern that may be useful for engineering review.

Predicted casualty crashes were given the greatest weight because the main goal of the analysis is to identify intersections with the highest expected casualty crash risk. Forecast stability was included to give more weight to intersections that consistently appeared in the Top 30 across repeated model runs, while crash profile strength was included as a smaller component to identify locations with particularly strong crash patterns.

The composite score was calculated as:

$$\text{Priority Score} = 0.50(\text{Forecast Risk}) + 0.30(\text{Forecast Stability}) + 0.20(\text{Crash Profile Strength})$$

It should be noted that the score is intended as a prioritization tool rather than a measure of absolute inspection safety. A high score means that an intersection should receive earlier engineering review.

Priority	Intersection	Score	Why Flagged
1	BOUNDARY RD @ GRANDVIEW HWY	90.20	#1 forecast and 97.8% stability; rear-end crashes 1.76x , nighttime crashes 1.38x , and heavy-vehicle involvement 1.26x
2	CLARK DR @ E 1ST AVE	71.02	High forecast and 90.2% stability; motorcycle involvement 1.96x , rear-end crashes 1.73x , and nighttime crashes 1.49x
3	MAIN ST @ TERMINAL AVE	69.87	High forecast and 92.6% stability; pedestrian involvement 1.86x , with elevated head-on and motorcycle involvement
4	E 41ST AVE @ KNIGHT ST	65.68	88.0% stability; heavy-vehicle involvement 1.90x , nighttime crashes 1.46x , and rear-end crashes 1.36x
5	GRANDVIEW HWY @ RUPERT ST	63.54	High forecast and 92.4% stability; head-on crashes 1.40x
6	KINGSWAY @ KNIGHT ST	63.46	High forecast and 91.0% stability; rear-end crashes 1.51x and nighttime crashes 1.41x
7	KINGSWAY @ VICTORIA DR	62.53	High forecast and 91.4% stability; nighttime crashes 1.46x , pedestrian involvement 1.35x , and head-on crashes 1.25x
8	E 41ST AVE @ VICTORIA DR	60.73	#2 casualty-crash forecast and 95.0% stability, despite no individual crash characteristic exceeding the 1.25x threshold

4.3 Conclusion

The analysis showed that historical crash data can be used to identify intersections with higher future casualty crash risk. The tuned Random Forest improved on the historical

baseline and was used to forecast 2023 casualty crashes, with forecast stability used to measure how consistent these predictions were.

Crash profiles were then used to identify patterns that may help explain what is happening at each intersection, and these results were combined into a final engineering priority score. This identified eight intersections as the strongest candidates for engineering review, providing both a way to prioritize where attention should go first and what crash patterns may be worth investigating.